

The Lain Compiler

Implementation Reference

Version 0.1 — Draft

April 23, 2026

Document status: Draft — not for normative use
Compiler version: Lain 1.0
Source base: src/ $\approx$ 15 000 LOC C99
Companion document: *The Lain Language Specification*
Date: April 23, 2026

This document describes the internal structure and algorithms of the Lain compiler reference implementation. It is intended for contributors, language implementors, and readers who want to understand how the compiler enforces the guarantees defined in the language specification. Where this document and the language specification differ, the language specification takes precedence.

Contents

I	Foundations	1
1	Architecture and Pipeline	3
1.1	Design philosophy	3
1.2	Compilation pipeline	3
1.3	Arena memory model	4
1.4	Source file structure	5
1.5	Naming conventions	6
1.6	Global state	7
1.7	Build instructions	7
2	Infrastructure	9
2.1	Primitive types	9
2.2	Arena allocator	9
2.3	File I/O	10
2.4	Panic assertions	11
2.5	ASCII classification	11
2.6	String type	11
3	Tokens and Lexer	13
3.1	Token representation	13
3.2	Token kinds	13
3.3	Lexer state machine	14
3.4	Special lexing cases	15
3.4.1	Whitespace and newlines	15
3.4.2	Numeric literals	15
3.4.3	String literals	16
3.4.4	Nested block comments	16
3.5	Keyword classification	16
3.6	Lookahead	16
II	Internal Representation	19
4	Abstract Syntax Tree	21
4.1	Identifier nodes	21
4.2	Type nodes	21
4.3	Declaration nodes	22
4.4	Statement nodes	24
4.5	Expression nodes	25
4.6	List nodes	27
4.7	Semantic annotation fields	27
4.8	Deep clone	27

5	Module System	29
5.1	Module records	29
5.2	Module name to path conversion	29
5.3	load_module algorithm	30
5.4	Cycle prevention	30
5.5	Diagnostic integration	31
III	Parsing	33
6	Parser Framework	35
6.1	Parser struct	35
6.2	Token advancement and normalization	35
6.3	Parser macros	36
6.4	Error reporting	36
6.5	Operator precedence	36
6.6	Numeric literal parsing	37
6.7	Path expression parsing	37
6.8	Parser file structure	37
7	Parser: Expressions	39
7.1	Entry point and structure	39
7.2	Pratt precedence climbing	39
7.3	Unary expressions	40
7.4	Primary expressions	40
7.4.1	Literals	40
7.4.2	Identifiers and member access	41
7.4.3	Case expressions	41
7.4.4	Builtin values	41
7.4.5	Parenthesised expressions	41
7.4.6	Array literals	41
7.5	Cast expressions	41
8	Parser: Statements and Declarations	43
8.1	Statement list parsing	43
8.2	Statement dispatch	43
8.3	Variable declarations	44
8.4	Assignment statements	44
8.5	Control-flow statements	44
8.5.1	if / elif / else	44
8.5.2	for	45
8.5.3	while	45
8.5.4	Match statements	45
8.6	Declaration parsing	45
IV	Semantic Analysis	47
9	Semantic Analysis: Orchestration	49
9.1	Two-phase analysis	49
9.2	sema_resolve_module entry point	49
9.3	Symbol table	50
9.4	Block scope	50

9.5	Walk-phase statement handler	51
9.6	Mutual recursion detection	51
10	Name Resolution	53
10.1	Identifier resolution	53
10.2	UFCS desugaring	53
10.3	Destructuring parameter expansion	53
10.4	Target platform constants	54
10.5	Scope building	54
11	Type Inference	55
11.1	Builtin type singletons	55
11.2	Integer widening	55
11.3	Type compatibility	55
11.4	Inference for each ExprKind	56
11.5	String type inference and generics	56
11.6	Parameter constraint propagation	56
12	Ownership and Linearity	57
12.1	Linear type classification	57
12.2	Linearity table	57
12.3	Field-level tracking	58
12.4	Consumption rules	58
12.5	Borrow interaction	58
12.6	Non-Lexical Lifetimes (NLL)	58
13	Borrow Checking and Regions	59
13.1	Regions	59
13.2	Borrow entries	59
13.3	Two-phase borrows	60
13.4	Aliasing rules	60
13.5	NLL last-use optimization	61
14	Value Range Analysis	63
14.1	Range representation	63
14.2	Saturating arithmetic	63
14.3	RangeTable	63
14.4	Constraint propagation	64
14.5	Condition-driven range refinement	64
14.6	In-guard table	64
15	Static Bounds Checking	65
15.1	Check algorithm	65
15.2	Safe fragment guarantee	65
15.3	Interaction with VRA	66
15.4	Interaction with unsafe	66
16	Pattern Matching Exhaustiveness	67
16.1	Wildcard arm	67
16.2	Enum exhaustiveness	67
16.3	Boolean exhaustiveness	67
16.4	Integer and other types	67

17 Termination Verification	69
17.1 Termination measure requirements	69
17.2 Non-negativity verification	69
17.3 Strict decrease verification	70
17.4 Accepted measure patterns	70
 V Code Generation	 71
18 Compile-Time Evaluation and Generics	73
18.1 Comptime environment	73
18.2 Expression evaluation	73
18.3 Generic function instantiation	73
18.4 Comptime-if	74
18.5 Type alias evaluation	74
 19 Code Generation	 75
19.1 Emit context	75
19.2 Output structure	75
19.3 Type emission	75
19.4 Enum emission	76
19.5 Defer mechanics	76
19.6 String literal initializers	76
 VI Standard Library and Testing	 77
20 Standard Library Implementation	79
20.1 std.c — C interoperability	79
20.2 std.io — Console I/O	79
20.3 std.fs — File system	79
20.4 std.math — Integer math	79
20.5 std.string — String utilities	80
20.6 std.option and std.result	80
 21 Testing Infrastructure	 81
21.1 Test file conventions	81
21.2 Test runner	81
21.3 Test categories	81
21.4 Invoking the test suite	82
 AST Node Reference	 83
.1 ExprKind	83
.2 StmtKind	83
.3 DeclKind	84
.4 TypeKind and OwnershipMode	85
 Diagnostic Catalogue	 87
.5 Linearity and ownership errors (E001–E010)	87
.6 Purity and mutability errors (E011)	88
.7 Type and call errors (E012–E015)	88
.8 Termination-measure errors (E080–E082)	89
.9 Parser errors (E100)	90

.10	VRA bounds diagnostics	90
.11	Diagnostic format	91
Pass Interactions		93
.12	Two-phase structure	93
.13	Shared global state	93
.14	Resolve phase interactions	94
.15	Walk phase interactions	94
.16	VRA and bounds interaction	95
.17	Linearity and borrows interaction	96
.18	Mutual recursion detection	96
.19	Pass dependency DAG	96
Declaration Flow		97
.20	Stage 1: Lexing	97
.21	Stage 2: Parsing	97
.22	Stage 3: Module loading	98
.23	Stage 4: Resolve phase	98
.24	Stage 5: Walk phase	99
.25	Stage 6: Mutual recursion check	99
.26	Stage 7: Code emission	99
.27	Per-kind summary	99
Index		101

Part I

Foundations

Chapter 1

Architecture and Pipeline

- 1 The Lain compiler is a single-pass, single-binary compiler written entirely in C99. Every subsystem is implemented as a header file (`.h`); the single translation unit `src/main.c` includes them all in dependency order. This design eliminates link-time symbol management, allows the C compiler to inline aggressively across subsystem boundaries, and makes the build reproducible with a single invocation of `gcc` or `clang`.
- 2 The compiler targets C99 as its sole output language. Lain source text is translated to a single C file that is compiled by the host C toolchain. This keeps the Lain compiler itself small and portable: it relies on the host compiler for platform ABI handling, optimisation, and linking.

1.1 Design philosophy

- 1 Four principles shape every architectural decision in the compiler.

Single translation unit.

All `.h` files are included into `src/main.c`. There is no separate compilation, no static library, and no linker step for the compiler itself. The include order is a direct expression of the dependency graph.

Three arenas, zero free.

Memory is allocated from three bump-pointer arenas (see section 1.3). No `malloc/free` calls appear in compiler logic; all memory is released in bulk when the compiler exits. This eliminates entire classes of memory-safety bugs and makes the compiler trivially valgrind-clean.

Two semantic phases, one pass each.

Semantic analysis runs in exactly two ordered phases per function: (1) name resolution and type inference (*resolve*), and (2) ownership, bounds, and purity checks (*walk*). The flag `sema_walk_phase` separates them. No phase reads results produced by a later phase.

C99 backend.

Emitting C99 instead of machine code delegates register allocation, instruction selection, and ABI handling to the host toolchain. The tradeoff is accepted compilation latency in exchange for a dramatically smaller compiler.

1.2 Compilation pipeline

- 1 Figure 1.1 shows the compiler pipeline. Each stage is a self-contained module; data flows forward through the pipeline.
- 2 The five major stages are:

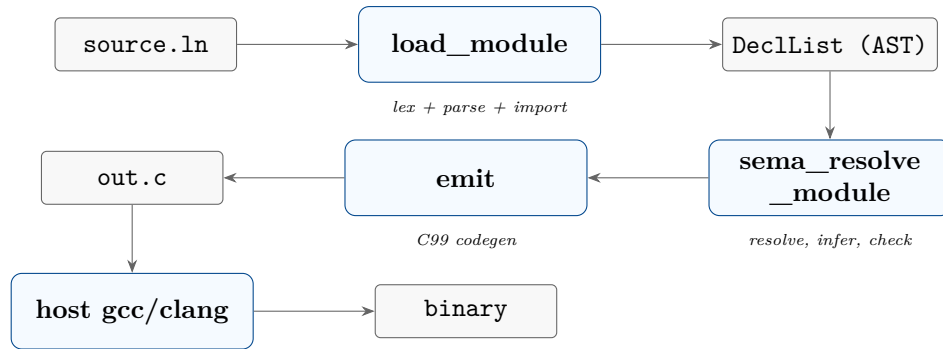


Figure 1.1: Lain compilation pipeline.

1. **Module loading** ([src/module.h](#)). Given the root module name, `load_module()` reads the source file from disk using `file_arena`, lexes it with `lexer_next()`, parses it with the recursive-descent parser, and recursively loads any `imported` modules. The result is a singly-linked `DeclList` of top-level declarations allocated in `ast_arena`.

2. **Semantic analysis** ([src/sema.h](#) and [src/sema/](#)). `sema_resolve_module()` runs two ordered phases on every function body:

- (a) *Resolve phase*: `sema_resolve_stmt()` populates identifier `->decl` pointers and infers expression types. UFCS desugaring, comptime generic substitution, and destructuring expansion happen here.
- (b) *Walk phase*: `walk_stmt()` runs VRA, bounds checking, ownership/linearity checking, purity checking, and bounded-while verification. The flag `sema_walk_phase` is `true` during this phase only.

After both phases complete for a function, `sema_check_function_linearity()` verifies that all owned values have been consumed or moved.

3. **Mutual recursion check** ([src/sema.h: 786](#)). After all functions are resolved, `sema_check_no_mutual_recursion()` performs DFS over the call graph and rejects any cycle that involves only `func` (pure) declarations.

4. **Code generation** ([src/emit.h](#) and [src/emit/](#)). `emit()` traverses the annotated `DeclList` and writes valid C99 to the output file. Struct types are forward-declared in topological order; constructors are synthesised for enum variants; slice types receive generated `typedef` names.

5. **Host compilation**. The compiler invokes no subprocess; it is the user's responsibility to compile the generated C file. The recommended invocation is documented in section 1.7.

1.3 Arena memory model

¹ The compiler uses three bump-pointer arenas declared in [src/main.c](#):

Arena variable	Contents	Lifetime
<code>file_arena</code>	Source text buffers, raw file bytes	Entire compiler run
<code>ast_arena</code>	All AST nodes (<code>Expr</code> , <code>Stmt</code> , <code>Decl</code> , <code>Type</code>), identifier strings, module records	Entire compiler run
<code>_sema_arena</code>	Symbol tables, <code>RangeTable</code> , borrow entries, in-guard entries, constraint lists	Entire compiler run

Invariant

No `free()` call is made on arena-allocated memory during compilation. Arenas are released only when the process exits. Compiler passes must never store a raw pointer to stack-allocated data inside arena-allocated structures.

- 2 The arena implementation ([src/utils/arena.h](#)) is a three-field struct `{beg, cur, end}`. Allocation advances `cur` by the requested size with optional alignment padding. When `ARENA_DEBUG` is enabled (the default), every push asserts that sufficient space remains, making out-of-memory conditions loud rather than silent.
- 3 The allocation macros are:

Macro	Usage
<code>arena_push(a, T)</code>	Allocate one <code>T</code>
<code>arena_push_many(a, T, n)</code>	Allocate <code>n</code> consecutive <code>T</code> values
<code>arena_push_aligned(a, T)</code>	Allocate one <code>T</code> with <code>alignof(T)</code> padding
<code>arena_push_many_aligned(a,T,n)</code>	Aligned array of <code>n</code> <code>T</code> values

1.4 Source file structure

- 1 Table 1.1 lists every header file in the compiler source tree together with its approximate line count and role.

Table 1.1: Compiler source files.

File	LOC	Role
src/main.c	115	Entry point, arena init, pipeline orchestration
src/token.h	340	<code>TokenKind</code> enum, <code>Token</code> struct
src/lexer.h	384	FSM lexer, <code>LexerState</code> , <code>lexer_next()</code>
src/ast.h	1032	All AST node types (<code>Expr</code> , <code>Stmt</code> , <code>Decl</code> , <code>Type</code>)
src/ast_clone.h	327	Deep-clone of AST subtrees (used by generics)
src/ast_print.h	508	<code>--dump-ast</code> debug printer
src/parser.h	–	Top-level parser include
src/parser/core.h	199	Parser state, lookahead, <code>expect()</code> , error recovery
src/parser/expr.h	403	Expression parser (Pratt precedence climbing)
src/parser/stmt.h	736	Statement parser
src/parser/decl.h	801	Declaration parser (<code>func</code> , <code>proc</code> , <code>struct</code> , <code>enum</code> , ...)
src/parser/type.h	–	Type expression parser
src/module.h	142	Module loading, import resolution, <code>ModuleNode</code>
src/sema.h	1066	Sema orchestrator, VRA, bounded-while verifier
src/sema/scope.h	146	Lexical scope (global/local symbol tables)
src/sema/resolve.h	987	Phase 1: name resolution, UFCS desugaring
src/sema/typecheck.h	1062	Phase 2 type inference, integer widening
src/sema/bounds.h	–	Bounds-check emission helpers
src/sema/ranges.h	594	VRA: <code>Range</code> , <code>RangeTable</code> , interval arithmetic
src/sema/region.h	538	Borrow checker: two-phase borrows, <code>BorrowTable</code>

(continued)

File	LOC	Role
src/sema/linearity.h	1398	Ownership/linearity checker, <code>LinearEnv</code>
src/sema/generic.h	216	Comptime generic instantiation
src/sema/comptime.h	321	Comptime expression evaluator
src/sema/exhaustiveness.h	315	Pattern exhaustiveness checker
src/sema/use_analysis.h	260	Use-before-init / unused-variable analysis
src/emit.h	–	Top-level emit include
src/emit/core.h	370	Emit state, <code>EmitCtx</code> , file writer helpers
src/emit/type_order.h	203	Topological sort of struct types
src/emit/decl.h	523	Declaration emitter (functions, structs, enums)
src/emit/stmt.h	627	Statement emitter
src/emit/expr.h	700	Expression emitter
src/emit/ctor.h	–	Enum constructor function emitter
src/emit/lain_header.h	325	Lain runtime header injected at top of output
src/utils/arena.h	124	Bump-pointer arena allocator
src/utils/common.h	–	Common primitive types (<code>isize</code> , <code>usize</code> , ...)
src/utils/ascii.h	241	ASCII classification helpers
src/utils/file.h	–	Platform file I/O (<code>file_read_all</code>)
src/utils/panic.h	–	<code>panic_if_msg()</code> assertion macro
src/utils/string.h	–	String utilities
src/utils/utf8.h	–	UTF-8 helpers
src/args.h	–	Command-line argument parsing

1.5 Naming conventions

- 1 Function and variable names in the compiler follow consistent prefixes that identify the owning subsystem.

Prefix	Subsystem
<code>lexer_</code>	Lexer (src/lexer.h)
<code>parse_</code>	Parser (src/parser/)
<code>sema_</code>	Semantic analysis orchestrator (src/sema.h)
<code>sema_resolve_ / resolve_</code>	Name resolution (src/sema/resolve.h)
<code>sema_infer_</code>	Type inference (src/sema/typecheck.h)
<code>range_</code>	Value range analysis (src/sema/ranges.h)
<code>borrow_</code>	Borrow checker (src/sema/region.h)
<code>linear_ / sema_check_</code>	Linearity checker (src/sema/linearity.h)
<code>emit_</code>	Code generator (src/emit/)
<code>arena_</code>	Arena allocator (src/utils/arena.h)
<code>mrec_</code>	Mutual-recursion DFS (src/sema.h)
<code>comptime_</code>	Comptime evaluator (src/sema/comptime.h)

- 2 Type names follow the pattern *SubsystemConcept*, e.g. `RangeTable`, `BorrowEntry`, `LinearEnv`, `InGuardEntry`. List nodes use the suffix `List` (e.g. `DeclList`, `StmtList`, `ExprList`).

1.6 Global state

- 1 The compiler maintains a small set of global variables that constitute the shared state visible across subsystem headers. These are declared (not merely declared `extern`) in the header that owns them; because all headers are included into a single translation unit, each global is defined exactly once.

Table 1.2: Compiler global state.

Variable	Type	Purpose
<code>sema_source_text</code>	<code>const char *</code>	Pointer into source buffer; used by <code>diagnostic_show_line</code>
<code>sema_source_file</code>	<code>const char *</code>	Source file name for error messages
<code>sema_in_guards</code>	<code>InGuardEntry *</code>	Head of in-guard linked list
<code>sema_decls</code>	<code>DeclList *</code>	Top-level declarations of current module
<code>sema_arena</code>	<code>Arena *</code>	Arena for sema-phase allocations
<code>sema_ranges</code>	<code>RangeTable *</code>	VRA range table for current function
<code>sema_in_unsafe_block</code>	<code>bool</code>	<code>true</code> inside an <code>unsafe {...}</code> block
<code>sema_walk_phase</code>	<code>bool</code>	<code>true</code> during walk phase only
<code>current_return_type</code>	<code>Type *</code>	Return type of function being analysed
<code>current_function_decl</code>	<code>Decl *</code>	<code>Decl</code> of function being analysed
<code>current_module_path</code>	<code>const char *</code>	Path of module being analysed
<code>emit_source_filename</code>	<code>const char *</code>	Source filename embedded in emit output

Note

The globals listed above are declared in `src/sema.h` and `src/emit/core.h`. Because they are file-scope statics or non-static definitions inside a single translation unit, no header guard is needed for uniqueness. The include order enforced by `src/main.c` ensures that consumer headers can reference them.

1.7 Build instructions

- 1 The compiler is built with a single compilation command. No build system is required.

```
1 gcc -std=c99 -Wall -Wextra -o lain src/main.c -I src
```

- 2 The generated C output is compiled by the host toolchain. The runtime header injected at the top of every generated file uses `#define` aliases for standard C library functions (e.g. `libc_printf`). When compiling generated output directly, pass the appropriate `-D` flags:

```
1 gcc -o out out.c -Dlibc_printf=printf -Dlibc_puts=puts -w
```

- 3 The `--dump-ast` flag instructs the compiler to print the parsed AST to `stdout` and exit without running `sema` or `emit`. This is useful for debugging parser changes.

```
1 ./lain --dump-ast source.ln
```


Chapter 2

Infrastructure

- 1 This chapter documents the utility layer that underpins all compiler subsystems. The infrastructure provides primitive types, arena allocation, file I/O, ASCII classification, string values, and the `panic` assertion mechanism. None of these utilities depends on Lain-specific data; they are general-purpose C99 building blocks found under `src/utils/`.

2.1 Primitive types

- 1 The file `src/utils/common/def.h` introduces fixed-width type aliases and size-annotated arithmetic types used throughout the compiler.

Alias	Underlying C type	Usage in compiler
<code>isize</code>	<code>ptrdiff_t</code> (signed)	Array indices, offsets, counts
<code>usize</code>	<code>size_t</code> (unsigned)	Allocation sizes
<code>iptr</code>	<code>intptr_t</code>	Pointer arithmetic
<code>uptr</code>	<code>uintptr_t</code>	Alignment calculations
<code>int8...int64</code>	<code>int8_t...int64_t</code>	Fixed-width signed integers
<code>uint8...uint64</code>	<code>uint8_t...uint64_t</code>	Fixed-width unsigned integers
<code>float32</code>	<code>float</code>	(unused in current compiler)
<code>float64</code>	<code>double</code>	(unused in current compiler)

- 2 The header also redefines `sizeof` and `alignof` to return `isize` and provides the `countof(a)` macro that divides the size of an array by the size of its first element.

```
1 #define sizeof(x)      (isize)sizeof(x)
2 #define alignof(x)     (isize)OPERATOR_ALIGNOF(x)
3 #define countof(a)     (sizeof(a) / sizeof(*(a)))
```

Rationale

Casting `sizeof` to `isize` eliminates signed/unsigned comparison warnings when loop indices (which are `isize`) are compared to sizes. This is a compile-time cast; no runtime overhead is incurred.

2.2 Arena allocator

- 1 The arena allocator (`src/utils/arena.h`) is a minimal bump-pointer allocator inspired by Chris Wellons's work. It provides the memory foundation for all three compiler arenas described in section 1.3.

- 2 The `Arena` struct contains three `char *` pointers:

Arena [src/utils/arena.h:23](#)

Field	Type	Semantics
<code>beg</code>	<code>char *</code>	Points to the first byte of the backing buffer.
<code>cur</code>	<code>char *</code>	Points to the next free byte (the bump pointer).
<code>end</code>	<code>char *</code>	Points one past the last byte of the buffer.

- 3 Allocation advances `cur` by the requested number of bytes. There is no `free`; the only reclamation operation is `arena_clear()`, which resets `cur` to `beg` and discards all previous allocations.

Algorithm

arena_push(arena, T)

1. Assert `size > 0` and `count > 0`.
2. Assert `cur + size*count <= end` (sufficient space).
3. Save `ptr = cur`.
4. Advance `cur += size * count`.
5. Return `ptr` cast to `T *`.

- 4 The aligned variant `_arena_push_aligned()` first computes the alignment padding as:

$$\text{padding} = -((\text{uptr}) \text{cur}) \& (\text{alignment} - 1)$$

This is the standard bitmask trick for aligning to a power-of-two boundary. The bump pointer is then advanced by `padding + size * count`.

- 5 When `ARENA_DEBUG` is defined to a non-zero value (the default), every push and alignment operation calls `panic_if_msg()` to abort with a diagnostic if any precondition is violated. Debug mode has no effect on the generated code path when all preconditions hold.

Invariant

`beg <= cur <= end` at all times. No pointer returned by an arena push is ever invalidated by a subsequent push, as the backing buffer is never reallocated.

2.3 File I/O

- 1 Source files are read into `file_arena` by `file_read_into_arena()` ([src/utils/file.h](#)). This function opens the file, queries its size, allocates `size + 1` bytes from the given arena, reads the entire file into that buffer, and NUL-terminates it. The one extra byte allows all string-based consumers (the lexer in particular) to treat the buffer as a C string without risk of reading past the end.
- 2 The returned `File` struct carries three fields:

File `src/utils/file.h:17`

Field	Type	Semantics
<code>handle</code>	<code>file</code>	Platform file handle (OS-specific typedef).
<code>size</code>	<code>isize</code>	Size of the file in bytes.
<code>contents</code>	<code>char *</code>	Pointer to arena-allocated NUL-terminated buffer.

- 3 If the file cannot be opened, `contents` is set to `NULL` and `size` to zero. The caller (`src/module.h`) checks for this condition and emits a diagnostic before aborting.

2.4 Panic assertions

- 1 The file `src/utils/panic.h` provides a family of assertion macros that print a structured error message to `stderr` and call `abort()`. These are used exclusively for *compiler-internal* invariants, not for user-visible error reporting.

Macro	Behaviour
<code>panic()</code>	Unconditional abort with file/line/function.
<code>panic_if(expr)</code>	Abort if <code>expr</code> is true.
<code>panic_msg(fmt, ...)</code>	Unconditional abort with formatted message.
<code>panic_if_msg(expr,fmt,...)</code>	Abort if <code>expr</code> is true, with message.

- 2 The macro output format reports the source file, line number, and function name of the failing assertion site. This distinguishes internal compiler bugs from user-visible semantic errors, which are reported via `fprintf(stderr, ...)` followed by `exit(1)`.

2.5 ASCII classification

- 1 The header `src/utils/ascii.h` provides inline classification predicates for ASCII characters. The lexer relies on these predicates to classify input bytes without depending on the locale-sensitive `<ctype.h>` functions, which are undefined for non-ASCII byte values.
- 2 The classifier functions include predicates for alphabetic characters (`ascii_is_alpha`), decimal digits (`ascii_is_digit`), hexadecimal digits, whitespace, and identifier-start / identifier-continue characters. Each is a `static inline` function that returns `bool`.

2.6 String type

- 1 The header `src/utils/string.h` defines a fat-pointer string struct:

```
1 typedef struct {
2     uint8 *data;
3     isize len;
4 } string;
```

- 2 This type is used in utility-level code that operates on raw byte sequences (hashing, comparison). It is distinct from the compiler's own `Id` type (see section 4.1), which is an arena-allocated `{name, length}` pair used for identifier tokens.

- 3 Two operations are provided: `string_hash(s)` computes a 64-bit hash of the string content using a multiplicative hash, and `string_equals(a, b)` compares two strings for equality using `memcmp`.
- 4 The `string_new(s)` macro wraps a C string literal:

```
1 #define string_new(s) { (uint8 *) s, lengthof(s) }
```

where `lengthof(s)` is `countof(s) - 1`, giving the string length without the NUL terminator.

Chapter 3

Tokens and Lexer

- 1 The Lain lexer converts a NUL-terminated source buffer into a flat stream of `Token` values. It is implemented in `src/lexer.h` and uses `src/token.h` for the token-kind enumeration and keyword-matching logic.
- 2 The lexer is zero-copy: every token holds a `const char *` pointer into the original source buffer together with a byte length. No new strings are allocated during lexing. The source buffer lives in `file_arena` for the lifetime of the compiler run.

3.1 Token representation

- 1 A `Token` is defined in `src/token.h` as:

```
1 typedef struct {
2     TokenKind    kind;
3     const char *start;
4     isize        length;
5 } Token;
```

Token `src/token.h:97`

Field	Type	Semantics
<code>kind</code>	<code>TokenKind</code>	Discriminant identifying the token category.
<code>start</code>	<code>const char *</code>	Pointer to the first byte of the lexeme in the source buffer.
<code>length</code>	<code>isize</code>	Byte length of the lexeme.

- 2 String tokens (`TOKEN_STRING_LITERAL`) are an exception: `start` points to the first byte *after* the opening double-quote and `length` excludes both delimiters. This makes string content immediately available without any stripping step in the parser.

3.2 Token kinds

- 1 The `TokenKind` enumeration (`src/token.h:6`) contains 95 members grouped as follows:

Group	Representative members
Control	TOKEN_EOF, TOKEN_NEWLINE, TOKEN_INVALID
Literals	TOKEN_NUMBER, TOKEN_FLOAT_LITERAL, TOKEN_STRING_LITERAL, TOKEN_CHAR_LITERAL
Identifiers	TOKEN_IDENTIFIER
Delimiters	TOKEN_L_PAREN, TOKEN_R_BRACE, TOKEN_L_BRACKET, ...
Operators	TOKEN_PLUS, TOKEN_MINUS, TOKEN_ASTERISK, ...
Compound ops	TOKEN_PLUS_EQUAL, TOKEN_EQUAL_EQUAL, TOKEN_SHIFT_LEFT, ...
Punctuation	TOKEN_DOT, TOKEN_DOT_DOT, TOKEN_DOT_DOT_EQUAL, TOKEN_COLON, ...
Comments	TOKEN_LINE_COMMENT, TOKEN_MULTILINE_COMMENT
Keywords	TOKEN_KEYWORD_IF through TOKEN_KEYWORD DECREASING

3.3 Lexer state machine

- 1 The `Lexer` struct holds two pointers:

Lexer [src/lexer.h:9](#)

Field	Type	Semantics
<code>text</code>	<code>const char *</code>	Pointer to the first byte of the source buffer (immutable).
<code>current</code>	<code>const char *</code>	Pointer to the next byte to be consumed.

- 2 The core function `lexer_next(Lexer *lexer)` ([src/lexer.h:51](#)) implements a single-function FSM. On entry, `token.start` is set to `current`. The FSM loops over bytes, transitions states via `LexerState`, and fires one of two macros:

```
1 #define SWITCH_STATE(_state) state = _state; break
2 #define RETURN_TOKEN(_kind) token.kind = _kind; \
3                             token.length = lexer->current - token
4                             .start; \
                             return token
```

`SWITCH_STATE` sets the next state and breaks the inner switch so the outer loop reads the next byte. `RETURN_TOKEN` computes the token length and returns.

- 3 The 22-member `LexerState` enum:

State	Entered after
STATE_START	Start of every token
STATE_IDENTIFIER	Any letter or <code>_</code>
STATE_NUMBER	Any decimal digit
STATE_SINGLE_QUOTE	<code>'</code>
STATE_DOUBLE_QUOTE	<code>"</code>
STATE_LINE_COMMENT	<code>//</code>
STATE_EQUAL	<code>=</code>
STATE_ANGLE_BRACKET_LEFT	<code><</code> (produces <code><</code> , <code><=</code> , <code><<</code>)
STATE_ANGLE_BRACKET_RIGHT	<code>></code> (produces <code>></code> , <code>>=</code> , <code>>></code>)
STATE_DOT	<code>.</code>
STATE_DOT_DOT	<code>..</code>
STATE_ASTERISK	<code>*</code>
STATE_SLASH	<code>/</code> (dispatches to <code>/</code> , <code>/=</code> , <code>//</code> , <code>/*</code>)
STATE_PERCENT	<code>%</code>
STATE_PLUS	<code>+</code>
STATE_MINUS	<code>-</code>
STATE_BANG	<code>!</code>
STATE_AMPERSAND	<code>&</code>
STATE_PIPE	<code> </code>
STATE_CARET	<code>^</code>

3.4 Special lexing cases

3.4.1 Whitespace and newlines

- Horizontal whitespace (`space` and `tab`) in `STATE_START` advances `token.start` silently, skipping the character without emitting a token. Vertical whitespace (`n` and `r`) returns `TOKEN_NEWLINE`. The parser uses newlines as statement terminators in contexts where a semicolon would otherwise be required.

3.4.2 Numeric literals

- Numeric scanning (`STATE_NUMBER`) handles four bases:
 - Decimal: sequences of `0–9` and underscore separators.
 - Hexadecimal: prefix `0x/0X` followed by hex digits and underscores.
 - Binary: prefix `0b/0B` followed by `0/1` and underscores.
 - Octal: prefix `0o/0O` followed by octal digits and underscores.
- A decimal point followed by digits produces `TOKEN_FLOAT_LITERAL`. If the decimal point is immediately followed by another dot (i.e. `..`), the dot is returned to the input and the integer token is returned, so `a..b` lexes correctly as an integer followed by the range operator.

3.4.3 String literals

- 4 In `STATE_DOUBLE_QUOTE`, a backslash causes the FSM to advance `current` one extra byte, consuming the escape without interpreting it. Escape decoding (`n`, `t`, `0`, etc.) is performed later in the parser. The resulting token's `start` points past the opening quote and `length` excludes the closing quote.

3.4.4 Nested block comments

- 5 Block comments (`/* ... */`) are handled inline in the `STATE_SLASH` case. A `nesting` counter supports arbitrarily deep nesting:

```
1 int nesting = 1;
2 while (nesting > 0) {
3     if (d == '/' && current[1] == '*') { nesting++; current
      += 2; }
4     else if (d == '*' && current[1] == '/') { nesting--; current
      += 2; }
5     else if (d == 0) break; /* EOF */
6     else current++;
7 }
```

3.5 Keyword classification

- 1 After an identifier token is scanned, `token_match_keyword(lexeme, len)` ([src/token.h:103](#)) tests whether it is a reserved keyword. The function dispatches on length, then uses `strncmp`:

Len	Keywords
2	<code>if</code> , <code>in</code> , <code>as</code> , <code>or</code>
3	<code>end</code> , <code>for</code> , <code>var</code> , <code>mov</code> , <code>use</code> , <code>and</code> , <code>pre</code>
4	<code>type</code> , <code>func</code> , <code>proc</code> , <code>expr</code> , <code>elif</code> , <code>else</code> , <code>case</code> , <code>post</code> , <code>true</code>
5	<code>break</code> , <code>false</code> , <code>macro</code> , <code>while</code> , <code>defer</code>
6	<code>import</code> , <code>export</code> , <code>extern</code> , <code>return</code> , <code>unsafe</code>
8	<code>continue</code> , <code>comptime</code>
9	<code>c_include</code> , <code>undefined</code>
10	<code>decreasing</code>

- 2 If no keyword matches, `TOKEN_IDENTIFIER` is returned. The length dispatch ensures that most identifiers avoid scanning the entire keyword table.

3.6 Lookahead

- 1 `lexer_peek(Lexer *lexer)` ([src/lexer.h:380](#)) provides non-consuming lookahead by copying the lexer state, calling `lexer_next()` on the copy, and returning the result without modifying the original:

```
1 Token lexer_peek(Lexer *lexer) {
2     Lexer temp = *lexer;
3     return lexer_next(&temp);
4 }
```


Because `Lexer` contains only two pointers, the copy is eight bytes on a 64-bit system and the peek has the same cost as a regular `lexer_next()`.

Part II

Internal Representation

Chapter 4

Abstract Syntax Tree

- 1 The Abstract Syntax Tree (AST) is defined entirely in `src/ast.h`. All AST nodes are allocated in `ast_arena` and live for the duration of the compiler run. The AST is built by the parser (chapter 6), annotated by the semantic analysis passes (chapter 9), and consumed by the code generator (chapter 19).
- 2 The AST uses four primary node types — `Decl`, `Stmt`, `Expr`, and `Type` — each discriminated by a kind enum and using a C `union` for per-kind payload. Each concrete node type that can appear in user source has a source-location pair (`line`, `col`) for diagnostics.

4.1 Identifier nodes

- 1 An `Id` is a zero-copy reference to an identifier lexeme:

```
1 typedef struct Id {  
2     isize      length;  
3     const char *name;  
4 } Id;
```

- 2 `name` points into the source buffer; no copy is made. The constructor `id(Arena*, isize, const char*)` allocates the `Id` struct in the given arena and stores the pointer and length. Identifier comparison uses `strncmp(a->name, b->name, a->length)` throughout the compiler; raw pointer equality is never used for identifier matching.

4.2 Type nodes

- 1 A `Type` node represents a Lain type expression. It carries an `OwnershipMode` that is orthogonal to the structural kind:

```
1 typedef enum { MODE_OWNED, MODE_SHARED, MODE_MUTABLE }  
    OwnershipMode;  
2 typedef enum { TYPE_SIMPLE, TYPE_ARRAY, TYPE_SLICE, TYPE_POINTER,  
3             TYPE_COMPTIME, TYPE_VARIANT, TYPE_META_TYPE }  
    TypeKind;
```

Type `src/ast.h:52`

Field	Type	Semantics
kind	TypeKind	Structural kind.
mode	OwnershipMode	Ownership semantics: MODE_OWNED (linear), MODE_SHARED (default), MODE_MUTABLE (borrow).
base_type	Id *	Base type name for TYPE_SIMPLE.
element_type	Type *	Inner type for arrays, slices, pointers, comptime.
array_len	isize	≥ 0 : fixed-length array; -1 : dynamic/run-time slice.
sentinel_str	const char *	Sentinel value string for TYPE_SLICE.
variant	Variant *	Payload descriptor for TYPE_VARIANT.

- 2 Seven constructor functions in [src/ast.h:492](#) build typed nodes with the correct defaults:

Constructor	Produces
type_simple(arena, base)	Named type with MODE_SHARED
type_array(arena, elem, len)	Fixed ($\text{len} \geq 0$) or dynamic ($\text{len} = -1$) array
type_slice(arena, elem, sent, ...)	Slice with sentinel
type_pointer(arena, elem)	Pointer type
type_move(arena, inner)	Copy of <code>inner</code> with MODE_OWNED
type_mut(arena, inner)	Copy of <code>inner</code> with MODE_MUTABLE
type_comptime(arena, base)	Comptime wrapper
type_meta_type(arena)	The <code>type</code> metatype

Invariant

OwnershipMode is a field on the Type struct, not a wrapper node. `type_move` and `type_mut` physically copy the inner struct and override the `mode` field. This means a type node cannot simultaneously carry two ownership modes; the last write wins.

4.3 Declaration nodes

- 1 A Decl represents a top-level or parameter declaration:

```

1 typedef struct Decl {
2     DeclKind kind;
3     union { DeclVariable variable_decl; DeclStruct struct_decl;
4             DeclEnum enum_decl; DeclFunction function_decl;
5             DeclImport import_decl; DeclCInclude c_include_decl;
6             DeclDestruct destruct_decl; DeclExternType
7                 extern_type_decl;
8             DeclTypeAlias type_alias_decl; } as;
9     isize line, col;
10 } Decl;

```

- 2 The 13-member DeclKind enumeration:

Kind	Payload	LaTeX source
DECL_VARIABLE	DeclVariable	Field or local (within params)
DECL_FUNCTION	DeclFunction	<code>func f(...)T { ... }</code>
DECL_PROCEDURE	DeclFunction	<code>proc p(...)T { ... }</code>
DECL_EXTERN_FUNCTION	DeclFunction	<code>extern func f(...)</code>
DECL_EXTERN_PROCEDURE	DeclFunction	<code>extern proc p(...)</code>
DECL_STRUCT	DeclStruct	<code>struct S { ... }</code>
DECL_ENUM	DeclEnum	<code>enum E { ... }</code>
DECL_IMPORT	DeclImport	<code>import foo.bar</code>
DECL_EVAL_IMPORT	DeclImport	Evaluated import
DECL_C_INCLUDE	DeclCInclude	<code>c_include "foo.h"</code>
DECL_DESTRUCT	DeclDestruct	Destructuring parameter
DECL_EXTERN_TYPE	DeclExternType	<code>extern type T</code>
DECL_TYPE_ALIAS	DeclTypeAlias	<code>type Name = Expr</code>

3 Key payload structs:

DeclVariable [src/ast.h:126](#)

Field	Type	Semantics
name	Id *	Variable/field name.
type	Type *	Type annotation (NULL if inferred).
in_field	Id *	Optional <code>in arr</code> annotation (proves index in bounds).
constraints	ExprList *	Equation-style constraints (e.g. <code>x int != 0</code>).
is_parameter	bool	True for function parameters.
is_mutable	bool	True if declared with <code>var</code> .

DeclFunction [src/ast.h:161](#)

Field	Type	Semantics
name	Id *	Function/procedure name.
params	DeclList *	Parameter declarations.
return_type	Type *	Return type.
body	StmtList *	Statement body.
pre_contracts	ExprList *	pre conditions.
post_contracts	ExprList *	post conditions.
return_constraints	ExprList *	Equation-style return constraints.
is_extern	bool	True for extern func/proc .
is_variadic	bool	True for variadic (...) parameter.

DeclEnum [src/ast.h:151](#)

Field	Type	Semantics
type_name	Id *	Enum type name.
variants	Variant *	Linked list of Variant nodes.

4.4 Statement nodes

- 1 A Stmt node represents a single statement:

```

1 typedef struct Stmt {
2     StmtKind kind;
3     isize line, col;
4     union { StmtVar var_stmt; StmtAssign assign_stmt; StmtExpr
      expr_stmt;
5         StmtIf if_stmt; StmtFor for_stmt; StmtUse use_stmt;
6         StmtMatch match_stmt; StmtReturn return_stmt;
7         StmtUnsafe unsafe_stmt; StmtWhile while_stmt;
8         StmtDefer defer_stmt; StmtComptimeIf comptime_if_stmt;
          } as;
9 } Stmt;

```

- 2 The 15-member StmtKind enumeration and payload structs:

Kind	Payload	Lain source
STMT_VAR	StmtVar	<code>var</code> x T = expr
STMT_ASSIGN	StmtAssign	x = expr
STMT_EXPR	StmtExpr	Expression statement
STMT_IF	StmtIf	<code>if</code> cond { ... } <code>else</code> { ... }
STMT_FOR	StmtFor	<code>for</code> i, v <code>in</code> iter { ... }
STMT_WHILE	StmtWhile	<code>while</code> cond [<code>decreasing</code> m] { ... }
STMT_MATCH	StmtMatch	<code>case</code> [&]expr { ... }
STMT_RETURN	StmtReturn	<code>return</code> expr
STMT_UNSAFE	StmtUnsafe	<code>unsafe</code> { ... }
STMT_DEFER	StmtDefer	<code>defer</code> stmt
STMT_USE	StmtUse	use x <code>as</code> alias
STMT_COMPTIME_IF	StmtComptimeIf	<code>comptime if</code> cond { ... }
STMT_CONTINUE	(none)	continue
STMT_BREAK	(none)	break

3 Notable payload fields:

StmtWhile [src/ast.h:256](#)

Field	Type	Semantics
cond	Expr *	Loop condition.
measure	Expr *	Termination measure (NULL for unbounded while; only allowed in <code>proc</code>).
body	StmtList *	Loop body.

StmtMatch [src/ast.h:268](#)

Field	Type	Semantics
value	Expr *	Scrutinee expression.
cases	StmtMatchCase *	Linked list of match arms.
is_borrowed	bool	True for <code>case &expr {...}</code> (non-consuming match).

4.5 Expression nodes

1 An Expr node represents an expression:

```
1 typedef struct Expr {
2     ExprKind kind;
```

```

3   isize    line, col;
4   union { /* 20 payload structs */ } as;
5   Type *type; /* annotated by sema */
6   Decl *decl; /* resolution target (if any) */
7   bool    is_global;
8 } Expr;

```

- 2 The `type` and `decl` fields are set to `NULL` by the parser and filled in by the semantic analysis passes. The 21-member `ExprKind` enumeration:

Kind	Payload	Lain source
EXPR_BINARY	ExprBinary	<code>a + b</code> , <code>a in arr</code> , ...
EXPR_UNARY	ExprUnary	<code>-x</code> , <code>not x</code> , ...
EXPR_IDENTIFIER	ExprIdentifier	<code>x</code>
EXPR_LITERAL	ExprLiteral	Integer literal
EXPR_FLOAT_LITERAL	ExprFloat	Float literal
EXPR_STRING	ExprString	<code>"hello"</code>
EXPR_CHAR	ExprChar	<code>'c'</code>
EXPR_MEMBER	ExprMember	<code>a.b</code>
EXPR_CALL	ExprCall	<code>f(a, b)</code>
EXPR_INDEX	ExprIndex	<code>arr[i]</code> , <code>arr[lo..hi]</code>
EXPR_RANGE	ExprRange	<code>lo..hi</code> , <code>lo..=hi</code>
EXPR_MOVE	ExprMove	<code>mov x</code>
EXPR_MUT	ExprMut	<code>mut x</code>
EXPR_CAST	ExprCast	<code>x as T</code>
EXPR_MATCH	ExprMatch	<code>case expr { ... }</code> (expression form)
EXPR_ARRAY_LITERAL	ExprArrayLiteral	<code>[a, b, c]</code>
EXPR_ANON_STRUCT	ExprAnonStruct	Anonymous struct literal
EXPR_ANON_ENUM	ExprAnonEnum	Anonymous enum
EXPR_TYPE	ExprType	Resolved type passed as value
EXPR_BUILTIN	ExprBuiltin	<code>@os</code> , <code>@arch</code>
EXPR_UNDEFINED	(none)	undefined sentinel value

ExprBinary [src/ast.h:356](#)

Field	Type	Semantics
<code>left</code>	<code>Expr *</code>	Left operand.
<code>op</code>	<code>TokenKind</code>	Operator <code>token</code> (e.g. <code>TOKEN_PLUS</code> , <code>TOKEN_KEYWORD_IN</code>).
<code>right</code>	<code>Expr *</code>	Right operand.

ExprCall [src/ast.h:393](#)

Field	Type	Semantics
callee	Expr *	Callee expression (typically <code>EXPR_IDENTIFIER</code> or <code>EXPR_MEMBER</code> after UFCS desugaring).
args	ExprList *	Linked list of argument expressions.

4.6 List nodes

- 1 Singly-linked list nodes connect AST nodes of the same category. Each list node carries one pointer to the contained node and a `next` pointer:

List type	Contains	Typical use
DeclList	Decl *	Top-level declarations; struct fields; function parameters
StmtList	Stmt *	Function/block bodies; branch bodies
ExprList	Expr *	Call arguments; match patterns; contract expressions
IdList	Id *	Destructuring name lists

- 2 List constructors (`decl_list`, `stmt_list`, `expr_list`, `id_list`) all follow the same pattern: allocate the list node from the arena, set the payload pointer, and set `next = NULL`. The parser appends to the tail of each list by scanning to the end; there is no tail pointer.

4.7 Semantic annotation fields

- 1 Three fields in `Expr` are `NULL` after parsing and filled by semantic analysis:

Field	Set by	Meaning
<code>->type</code>	<code>sema_infer_expr()</code>	Resolved <code>Type *</code> of the expression.
<code>->decl</code>	<code>sema_resolve_expr()</code>	<code>Decl *</code> this identifier/call refers to.
<code>->is_global</code>	<code>sema_resolve_expr()</code>	True if the identifier is a top-level symbol.

- 2 The code generator relies on these annotations to emit correct C output. It is a compiler-internal invariant that `->type` is non-`NULL` for every `Expr` node that reaches the emitter, unless the expression is of kind `EXPR_UNDEFINED` or `EXPR_TYPE`.

4.8 Deep clone

- 1 [src/ast_clone.h](#) implements deep-copy functions for all major AST node types (`clone_decl`, `clone_stmt`, `clone_expr`, `clone_type`). These are used exclusively by the `comptime` generic instantiation pass (chapter 18) to produce per-instantiation copies of generic function bodies before substituting type parameters. No other pass calls the clone functions.

Chapter 5

Module System

- 1 The Lain module system translates a flat dotted module name (e.g. `foo.bar`) to a filesystem path (`foo/bar.ln`), reads and parses the file, and recursively expands `import` declarations in-place in the AST. The entire mechanism is implemented in `src/module.h`.

5.1 Module records

- 1 A `ModuleNode` records every module that has been loaded during a compiler run:

`ModuleNode` [src/module.h:22](#)

Field	Type	Semantics
<code>name</code>	<code>char *</code>	Canonical dotted name (" <code>foo.bar</code> ").
<code>decls</code>	<code>DeclList *</code>	Top-level declaration list for this module.
<code>source_text</code>	<code>const char *</code>	NUL-terminated source buffer (for diagnostic line display).
<code>source_file</code>	<code>const char *</code>	Filesystem path (for error messages).
<code>next</code>	<code>ModuleNode *</code>	Next node in the global linked list.

- 2 The global `loaded_modules` variable is the head of a singly-linked list of all loaded modules. Module records are allocated in `ast_arena`; the name string is arena-copied so the record does not hold a pointer into a stack-allocated buffer.

5.2 Module name to path conversion

- 1 `module_name_to_path(mod, out, cap)` ([src/module.h:63](#)) translates a dotted module name to a filesystem path by replacing every `.` with `/` and appending the `.ln` extension:

Algorithm

```
For each character c in mod:
  if c == '.': write /
  else: write c
Append ".ln".
```

- 2 Paths are capped at 255 characters (the fixed `path[256]` buffer). The compiler aborts if the module name is longer, which cannot occur for reasonable module names.

5.3 load_module algorithm

- 1 `load_module(file_arena, ast_arena, modname)` ([src/module.h:78](#)) is the single entry point for module loading. It is called recursively for each import dependency.

Algorithm

load_module(*file_arena*, *ast_arena*, *modname*):

1. If `modname` is already in `loaded_modules`, return `NULL`. (Already loaded modules are silently skipped to prevent duplicate symbols.)
2. Translate `modname` to a filesystem path via `module_name_to_path`.
3. Read the file into *file_arena* using `file_read_into_arena`. Abort with an error if the file is missing.
4. Create a `Lexer` pointing at the source buffer and a `Parser` wrapping the lexer.
5. Call `parse_module(ast_arena, &parser)` to build a `DeclList`.
6. Walk the top-level `DeclList`. For each `DECL_IMPORT` node:
 - (a) Recursively call `load_module` for the imported module name.
 - (b) If the recursive call returns a non-`NULL` `DeclList`, splice it in-place into the current list: the import node is removed and the imported declarations are inserted at that position.
7. Call `record_module` to save the module record in the linked list.
8. Return the (now-spliced) `DeclList`.

- 2 The splice-in-place strategy means that after `load_module` returns, the root `DeclList` passed to `sema_resolve_module` already contains the declarations of all imported modules in a single flat list. There is no separate per-module processing in `sema`; all top-level symbols are globally visible within the flat list.

Rationale

Splicing imports into a single flat list simplifies name resolution: there is no module namespace or scope stack at the top level. Every top-level symbol is accessible from any function in the program. This is sufficient for the current Lain compiler which does not expose separate compilation or import qualified access.

5.4 Cycle prevention

- 1 Before loading a module, `module_already_loaded()` performs a linear scan of `loaded_modules`. If the module is already present, `NULL` is returned immediately. When the caller splices the result, a `NULL` return means “already loaded; nothing to splice”, so the import node is silently dropped.

Invariant

Every distinct module name appears at most once in `loaded_modules`. Mutually importing modules terminate because the second call to `load_module("A")` from inside module B (which was loaded first from A) finds A already registered and returns `NULL`.

5.5 Diagnostic integration

- 1 `record_module` stores both the source text and the filesystem path in the `ModuleNode`. Before running semantic analysis on a module, `sema_resolve_module()` calls `find_module(module_path)` and copies the pointers into the globals `sema_source_text` and `sema_source_file` ([src/sema.h:841](#)). These are then available to `diagnostic_show_line()` when any semantic error is reported.
- 2 The `path` variable used as `source_file` is the local `char path[256]` buffer inside `load_module`. Its address is stored in the module record, which means the module record is invalidated if the stack frame unwinds. This is safe only because `record_module` is called at the end of `load_module` before the function returns, and the caller uses the pointer while the frame is still live, or — after the refactor that allocates records in `ast_arena` — the pointer points to a fixed path variable whose lifetime is bounded by the module loading phase.

Note

A future improvement would copy `path` into `ast_arena` before storing it, eliminating the dependency on stack lifetime.

Part III

Parsing

Chapter 6

Parser Framework

- 1 The Lain parser is a hand-written, single-pass, recursive-descent parser. It is split across four header files under `src/parser/`, each responsible for one category of construct. The shared infrastructure — the `Parser` struct, lookahead management, error reporting, and operator precedence — lives in `src/parser/core.h`.

6.1 Parser struct

- 1 The `Parser` struct is the sole mutable state during parsing:

`Parser` `src/parser/core.h:6`

Field	Type	Semantics
<code>lexer</code>	<code>Lexer *</code>	Pointer to the lexer (owns the scan position).
<code>token</code>	<code>Token</code>	Current lookahead token (one-token buffer).
<code>line</code>	<code>isize</code>	Current source line (incremented on <code>TOKEN_NEWLINE</code>).
<code>column</code>	<code>isize</code>	Current source column (advanced by token length).

- 2 The parser maintains exactly one token of lookahead in `token`. Before any parse function is called, the caller must have already consumed the previous token so that `token` holds the next unconsumed token. The invariant is: *at the entry of every parse function, `parser->token` is the first unconsumed token relevant to that function.*

6.2 Token advancement and normalization

- 1 `_parser_advance(parser)` (`src/parser/core.h:37`) is the single function that advances the lookahead. It:
 1. Calls `lexer_next()` in a loop, discarding `TOKEN_LINE_COMMENT` and `TOKEN_MULTILINE_COMMENT` tokens.
 2. Updates `parser->line` and `parser->column` based on the raw token.

3. Normalizes `TOKEN_NEWLINE` to `TOKEN_EOL`, the canonical end-of-line token used by the rest of the parser.
 4. Rejects `TOKEN_SEMICOLON` with error **[E100]**: semicolons are not allowed in Lain.
- 2 The normalization step ensures that all statement-termination logic uses `TOKEN_EOL` exclusively. No parser function ever tests for `TOKEN_NEWLINE` or `TOKEN_SEMICOLON`.

6.3 Parser macros

- 1 A family of macros in [src/parser/core.h:22](#) provides convenient shorthands:

Macro	Expands to
<code>parser_match(k)</code>	<code>parser->token.kind == k</code>
<code>parser_advance()</code>	<code>_parser_advance(parser)</code>
<code>parser_expect(k, msg)</code>	error if <code>!parser_match(k)</code>
<code>parser_error(msg)</code>	unconditional error
<code>parser_is_eol()</code>	<code>parser_match(TOKEN_EOL)</code>
<code>parser_skip_eol()</code>	advance while EOL or comment
<code>parser_expect_eol(msg)</code>	error if not at EOL

- 2 `parser_expect(k, msg)` is the most commonly used macro. It asserts that the current token has kind `k`; if not, it calls `_parser_error`, which prints the line/column and the message and exits.

6.4 Error reporting

- 1 Parse errors are reported via `_parser_error(parser, msg)`, which writes to `stderr` in the format:

```
1 [E100] Error Ln %li, Col %li: %message%
```

- 2 All parse errors use the catch-all code **[E100]**. Semantic error codes (E001–E082) are distinct and emitted by sema passes. After printing, the function calls `exit(1)`. There is no error recovery; the first parse error terminates the compiler.

6.5 Operator precedence

- 1 Binary operator precedence is defined by `get_precedence(TokenKind)` ([src/parser/core.h:89](#)). Higher numbers bind more tightly. The table aligns with §8.7 of the Lain language specification:

Level	Operators	Associativity
10	* / %	Left
9	+ -	Left
8	<< >>	Left
7	< <= > >= in	Left
6	== !=	Left
5	&	Left
4	^	Left
3		Left
2	and	Left
1	or	Left

- 2 Operators not in the table return -1 from `get_precedence`, which terminates the Pratt climbing loop in the expression parser (see section 7.2).
- 3 The **in** operator is at the same precedence level as the comparison operators (level 7). This allows `idx in arr and arr[idx] != 0` to parse as `(idx in arr) and (arr[idx] != 0)`, where the left-hand **in** guard proves the right-hand index safe.

6.6 Numeric literal parsing

- 1 `parse_numeric_literal(start, length)` ([src/parser/core.h:179](#)) converts a lexer token into a `long long`. It strips underscore separators (e.g. `1_000_000`), then dispatches on the base prefix:
 - `0b/0B` prefix: calls `strtoll(buf+2, NULL, 2)`.
 - `0o/0O` prefix: calls `strtoll(buf+2, NULL, 8)`.
 - `0x/0X` prefix: calls `strtoll(buf, NULL, 16)`.
 - No prefix: calls `strtoll(buf, NULL, 10)`.
- 2 The function uses a fixed 64-byte buffer for the stripped literal. A literal longer than 63 characters after stripping underscores will be silently truncated; this is acceptable given that all integer types in Lain fit within 64 bits.

6.7 Path expression parsing

- 1 `parse_path_expr(arena, parser)` ([src/parser/core.h:147](#)) parses a dotted identifier path (e.g. `std.io.println`) as a sequence of `EXPR_MEMBER` nodes. It starts with an `EXPR_IDENTIFIER` and extends it with member accesses for each dot-separated component. This helper is shared between use statement parsing and module path parsing.

6.8 Parser file structure

- 1 The parser is split into four files included transitively through [src/parser.h](#):

File	Content
src/parser/core.h	Parser struct, advance, macros, precedence
src/parser/expr.h	Expression parser (Pratt climbing)
src/parser/stmt.h	Statement parser
src/parser/decl.h	Declaration parser (<code>parse_module</code>)
src/parser/type.h	Type expression parser

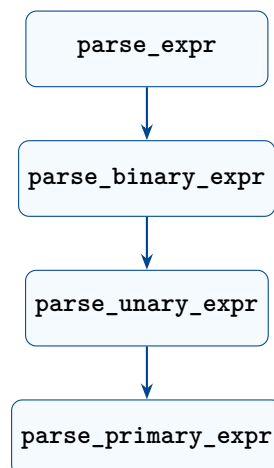
Chapter 7

Parser: Expressions

- 1 Expression parsing is implemented in [src/parser/expr.h](#). The parser uses a standard Pratt precedence-climbing algorithm for binary expressions, with a separate recursive-descent layer for unary operators and primary expressions. The entry point is `parse_expr`.

7.1 Entry point and structure

- 1 `parse_expr(arena, parser)` captures the current source location, calls `parse_binary_expr(arena, parser, 0)`, and stamps the resulting node's `line/col` fields. The source location is captured before any token is consumed, so it refers to the start of the expression.
- 2 The call hierarchy is:



7.2 Pratt precedence climbing

- 1 `parse_binary_expr(arena, parser, min_prec)` ([src/parser/expr.h:27](#)) implements left-associative precedence climbing:

Algorithm

parse_binary_expr(arena, parser, min_prec):

1. Parse a unary expression as *left*.
2. If the current token is **as**, parse a cast type and wrap *left*.
3. Loop:
 - (a) Let `prec = get_precedence(token.kind)`.

- (b) If $prec < min_prec$, break.
 - (c) Consume the operator token.
 - (d) Recurse: $right = \text{parse_binary_expr}(\text{arena}, \text{parser}, prec+1)$.
 - (e) Build `EXPR_BINARY(op, left, right)` and assign to *left*.
 - (f) If the next token is `as`, parse a cast and wrap *left*.
4. Return *left*.

- 2 The recursive call with $prec+1$ produces left-associativity: operators at the same precedence level associate to the left because the right operand only claims operators of strictly higher precedence.
- 3 The `as` cast check after each sub-expression is required because `as` is a postfix operator with higher binding than binary operators. The check after the initial unary parse handles `a as T op b`; the check inside the loop handles `a op b as T op c`.

7.3 Unary expressions

- 1 `parse_unary_expr` ([src/parser/expr.h:60](#)) handles all prefix unary operators:

Token	AST node	Meaning
<code>-</code>	<code>EXPR_UNARY(TOKEN_MINUS, right)</code>	Arithmetic negation
<code>!</code>	<code>EXPR_UNARY(TOKEN_BANG, right)</code>	Logical not
<code>&</code>	<code>EXPR_UNARY(TOKEN_AMPERSAND, right)</code>	Address-of / borrow
<code>~</code>	<code>EXPR_UNARY(TOKEN_TILDE, right)</code>	Bitwise complement
<code>*</code>	<code>EXPR_UNARY(TOKEN_ASTERISK, right)</code>	Pointer dereference
<code>mov</code>	<code>EXPR_MOVE(right)</code>	Explicit ownership transfer
<code>var</code>	<code>EXPR_MUT(right)</code>	Mutable borrow

- 2 Unary operators are right-recursive: `parse_unary_expr` calls itself for the operand. This allows chaining (`--x` if supported, `!not y`, etc.) and correctly handles `mov *ptr`.

7.4 Primary expressions

- 1 `parse_primary_expr` ([src/parser/expr.h:99](#)) handles all atom-level and postfix constructs.

7.4.1 Literals

- 2 Integer literals (`TOKEN_NUMBER`) are parsed with `parse_numeric_literal` and stored as `EXPR_LITERAL` with an `int` value. Float literals use `strtod` and produce `EXPR_FLOAT_LITERAL`. String literals produce `EXPR_STRING` with the content pointer already stripped of delimiters (see section 3.4.3). Character literals produce `EXPR_CHAR` with a decoded `char` value; escape sequences (`n`, `t`, `,`, `'`) are decoded at parse time.

7.4.2 Identifiers and member access

- 3 An `TOKEN_IDENTIFIER` produces an `EXPR_IDENTIFIER` node. The parser then checks for postfix operators in a loop:
 - `.` followed by an identifier: wraps in `EXPR_MEMBER`.
 - `[` followed by an expression: produces `EXPR_INDEX`. If the subscript contains `..` or `..=`, the index expression is an `EXPR_RANGE`.
 - `(` followed by arguments: produces `EXPR_CALL`.
- 4 Postfix operators are left-associative and bind more tightly than any binary operator, so `a.b[i](x)` parses as `((a.b)[i])(x)`.

7.4.3 Case expressions

- 5 A `case` keyword inside an expression context initiates a match expression. The optional `&` prefix marks the scrutinee as borrowed (`is_borrowed = true`). Arms are parsed as `pattern => body` pairs inside a brace-delimited block. The `else` keyword introduces the wildcard arm.

7.4.4 Builtin values

- 6 `@os` and `@arch` produce `EXPR_BUILTIN` nodes carrying the appropriate `BuiltinKind` discriminant. These are evaluated at comptime by `sema_eval_comptime_expr()` in `src/sema/comptime.h`.

7.4.5 Parenthesised expressions

- 7 A parenthesised expression `(expr)` consumes `(`, calls `parse_expr` recursively, then expects `)`. The result is the inner expression with no wrapper node.

7.4.6 Array literals

- 8 `[` initiates an array literal. Elements are comma-separated and delimited by `]`. The result is `EXPR_ARRAY_LITERAL` with an `ExprList` of elements.

7.5 Cast expressions

- 1 The `as` keyword desugars to `EXPR_CAST(expr, target_type)`. It is treated as a postfix operator rather than a binary operator; this means it does not participate in the standard precedence table and is checked explicitly after each sub-expression during Pratt climbing. The effect is that `as` binds tighter than all binary operators: `a + b as T` parses as `a + (b as T)`.

Rationale

Treating `as` as postfix prevents the ambiguity that arises if it were a binary operator with a precedence level: `a + b as T + c` would be ambiguous as to whether `T + c` is a binary expression or part of the type.

Chapter 8

Parser: Statements and Declarations

- 1 Statement parsing is in [src/parser/stmt.h](#); declaration (top-level) parsing is in [src/parser/decl.h](#). Both files use the framework described in chapter 6.

8.1 Statement list parsing

- 1 `parse_stmt_list(arena, parser)` ([src/parser/stmt.h:72](#)) accumulates statements into a `StmtList` until it encounters `}` or `TOKEN_EOF`. After each statement it requires an end-of-line token (`TOKEN_EOL`), then skips any subsequent blank lines.

Algorithm

parse_stmt_list(*arena*, *parser*):

1. Skip leading EOL tokens.
2. While not at `}` or EOF:
 - (a) Parse one statement via `parse_stmt`.
 - (b) Append to the list.
 - (c) If the next token is `}`, stop.
 - (d) Require EOL (newline terminates the statement).
 - (e) Skip further EOL tokens.
3. Return the list.

8.2 Statement dispatch

- 1 `parse_stmt` ([src/parser/stmt.h](#)) dispatches on the current token to the appropriate parse function:

Token	Parse function	Stmt kind
var	parse_var_stmt	STMT_VAR
if	parse_if_stmt	STMT_IF
elif	(handled inside if)	STMT_IF (chained)
for	parse_for_stmt	STMT_FOR
while	parse_while_stmt	STMT_WHILE
case	parse_match_stmt	STMT_MATCH
return	(inline)	STMT_RETURN
unsafe	parse_unsafe_stmt	STMT_UNSAFE
defer	parse_defer_stmt	STMT_DEFER
use	parse_use_stmt	STMT_USE
continue	parse_continue_stmt	STMT_CONTINUE
break	parse_break_stmt	STMT_BREAK
comptime	parse_comptime_stmt	STMT_COMPTIME_IF
identifier	parse_assign_stmt or parse_decl_stmt	STMT_ASSIGN/STMT_VAR
otherwise	parse_expr_stmt	STMT_EXPR

8.3 Variable declarations

- 1 There are two syntactic forms for variable declarations:
 1. **Explicit var:** `var name [Type] = expr.` The `var` keyword is required for mutable bindings. `parse_var_stmt` expects the `var` keyword, then parses name, optional type annotation, and required initializer.
 2. **Implicit:** `name Type = expr.` When `parse_stmt` sees an identifier that is not a known keyword, it calls `parse_decl_stmt` which treats the identifier as the variable name, followed by a type annotation and an initializer.
- 2 Both forms require an initializer; `= undefined` is allowed as a bypass marker. Immutable declarations (`name Type = undefined`) are rejected at parse time.

8.4 Assignment statements

- 1 An assignment is parsed by `parse_assign_stmt` when the current token is an identifier and the lookahead after parsing a target expression is `=` (or a compound assignment operator like `+=`, `-=`, etc.). The target is an arbitrary left-hand side expression (identifier, member access, index expression). Compound assignments are desugared by the parser into `target = target op rhs` before AST construction.

8.5 Control-flow statements

8.5.1 if / elif / else

- 1 `parse_if_stmt` parses the condition expression, the `{` block, then checks for `elif` or `else`. An `elif` is desugared into a nested `STMT_IF` in the else branch. The else branch is either a `StmtList` (from `else { ... }`) or the single nested `STMT_IF` from `elif`.

8.5.2 for

- 2 **for** [idx,] val **in** iterable { body } is parsed by `parse_for_stmt`. The optional index name is separated from the value name by a comma. The iterable is an arbitrary expression. The body is a statement list.

8.5.3 while

- 3 `parse_while_stmt` parses **while** cond [**decreasing** measure] { body }. After the condition, the parser checks for the `TOKEN_KEYWORD DECREASING` token. If present, the next expression is the termination measure, stored in `StmtWhile.measure`; otherwise `measure` is `NULL`.
- 4 A `NULL` measure means the while loop is unbounded and is only legal in a **proc** (procedure) context. The sema pass enforces this constraint.

8.5.4 Match statements

- 5 **case** [&] scrutinee { arms } is parsed by `parse_match_stmt`. The optional **&** marks the match as borrowed (`is_borrowed = true`). Each arm is a comma-separated list of patterns followed by `=>` and a body. The wildcard arm uses the **else** keyword as the pattern.

8.6 Declaration parsing

- 1 `parse_module(arena, parser)` in `src/parser/decl.h` is the top-level entry point. It calls `parse_decl` in a loop until `TOKEN_EOF`.
- 2 Top-level declarations parsed by `parse_decl`:

Keyword	Decl kind	Payload
func	<code>DECL_FUNCTION</code>	<code>DeclFunction</code>
proc	<code>DECL_PROCEDURE</code>	<code>DeclFunction</code>
struct	<code>DECL_STRUCT</code>	<code>DeclStruct</code>
enum	<code>DECL_ENUM</code>	<code>DeclEnum</code>
import	<code>DECL_IMPORT</code>	<code>DeclImport</code>
extern	<code>DECL_EXTERN_*</code>	<code>DeclFunction</code>
c_include	<code>DECL_C_INCLUDE</code>	<code>DeclCInclude</code>
type	<code>DECL_TYPE_ALIAS</code>	<code>DeclTypeAlias</code>

- 3 Function and procedure parsing share the same `DeclFunction` payload. The body parser calls `parse_stmt_list` for the block. Optional `pre/post` contract clauses are parsed after the return type and before the body block.
- 4 Struct field declarations inside a **struct** body are parsed as a list of `DECL_VARIABLE` nodes. Each field may carry an optional **in** field annotation (for bounds-proven indices) and equation-style constraints.
- 5 Enum declarations parse a list of variants. Each variant is either a simple identifier or an identifier followed by a brace-delimited list of fields (forming an algebraic data type variant).

Part IV

Semantic Analysis

Chapter 9

Semantic Analysis: Orchestration

- 1 The semantic analysis subsystem transforms a raw parse tree into a fully-annotated, safety-verified AST ready for code generation. The orchestrator lives in `src/sema.h`; the individual passes live in the headers under `src/sema/`.

9.1 Two-phase analysis

- 1 Semantic analysis runs in two ordered phases for each function body:

Phase 1 — Resolve.

`sema_resolve_stmt()` and `sema_resolve_expr()` populate `->decl` pointers on `EXPR_IDENTIFIER` and `EXPR_CALL` nodes, expand UFCS calls, instantiate comptime generics, and expand destructuring parameters. The flag `sema_walk_phase` is `false` during this phase.

Phase 2 — Walk.

`walk_stmt()` runs type inference (`sema_infer_expr`), value range analysis (VRA), bounds checking, and bounded-while verification. The flag `sema_walk_phase` is `true` during this phase. Checks that must only run during walk (e.g. array bounds) test this flag before emitting errors.

- 2 The separation prevents resolve-phase operations from seeing walk-phase state (e.g. range information that has not yet been computed) and prevents walk-phase operations from emitting spurious errors on UFCS-desugared calls that are not yet complete.

9.2 `sema_resolve_module` entry point

- 1 `sema_resolve_module(decls, module_path, arena)` (`src/sema.h:833`) is called once per compilation with the flat declaration list produced by `load_module`.

Algorithm

`sema_resolve_module`(*decls*, *module_path*, *arena*):

1. Store `arena` in `sema_arena`; build a fresh `RangeTable`.
2. Retrieve the source text and file from the `ModuleNode` record.
3. Clear the global symbol table and populate it via `sema_build_scope`.
4. For each `DECL_FUNCTION` or `DECL_PROCEDURE` in *decls*:
 - (a) Clear the local symbol table.

- (b) Insert parameter symbols (including destructuring expansion).
 - (c) Apply inline parameter constraints and pre-contracts to the range table.
 - (d) **Phase 1:** run `sema_resolve_stmt` on the entire body.
 - (e) **Phase 2:** set `sema_walk_phase = true`, run `walk_stmt` on the entire body, then clear the flag.
 - (f) Run `sema_check_function_linearity(d)` to verify ownership.
 - (g) Clear the local symbol table.
5. Run `sema_check_no_mutual_recursion(decls)`.

9.3 Symbol table

- 1 The symbol table is implemented in `src/sema/scope.h` as two separate open-hash tables with 4096 buckets:

Table	Contents
<code>sema_globals[4096]</code>	Top-level declarations: functions, structs, enums, type aliases
<code>sema_locals[4096]</code>	Per-function: parameters, var declarations

- 2 Lookup (`sema_lookup`) checks locals first, then globals, so function parameters shadow top-level symbols.

Symbol `src/sema/scope.h:33`

Field	Type	Semantics
<code>name</code>	<code>char *</code>	Raw identifier string (arena copy).
<code>c_name</code>	<code>char *</code>	C-mangled identifier for emit.
<code>type</code>	<code>Type *</code>	Symbol type (NULL before inference).
<code>decl</code>	<code>Decl *</code>	Originating declaration (NULL for implicit locals).
<code>is_global</code>	<code>bool</code>	True if in <code>sema_globals</code> .
<code>is_mutable</code>	<code>bool</code>	True if declared var .
<code>next</code>	<code>Symbol *</code>	Hash-chain next.

- 3 The hash function is DJB2 (`sema_hash`) modulo 4096. All `Symbol` nodes are allocated in `sema_arena`. The “clear” operations (`sema_clear_globals`, `sema_clear_locals`) reset bucket pointers to NULL; arena memory is not released.

9.4 Block scope

- 1 Within a function body, block scope is maintained via a linked list of `ScopeFrame` nodes. Each frame saves a full snapshot of the 4096-pointer `sema_locals` array.

```

1 static void sema_push_scope(void) {
2     ScopeFrame *frame = arena_push_aligned(sema_arena, ScopeFrame)
    ;

```

```

3     memcpy(frame->saved_locals, sema_locals, sizeof(sema_locals));
4     frame->parent = scope_stack;
5     scope_stack = frame;
6 }
7 static void sema_pop_scope(void) {
8     memcpy(sema_locals, scope_stack->saved_locals, sizeof(
9         sema_locals));
10    scope_stack = scope_stack->parent;
11 }

```

- 2 Symbols declared inside a block (STMT_VAR, STMT_FOR index) are inserted into `sema_locals` during the block. On pop, the previous snapshot is restored, making the block-local symbols invisible to subsequent code.

Invariant

Every `sema_push_scope()` call is matched by exactly one `sema_pop_scope()` call. The orchestrator wraps each function body pass in push/pop pairs; individual statement handlers do the same for their sub-blocks.

9.5 Walk-phase statement handler

- 1 `walk_stmt(s)` ([src/sema.h:412](#)) is the walk-phase driver. For each statement kind it:
 - Calls `sema_infer_expr` on all sub-expressions to set their `->type` fields.
 - Updates the `RangeTable` via `range_set` and `sema_apply_constraint`.
 - Pushes/pops in-guards for `STMT_IF` and `STMT_WHILE` conditions containing `in` expressions.
 - Calls `sema_verify_bounded_while` for `while` loops that carry a termination measure.
 - Recursively processes sub-statement lists within push/pop scope pairs.
- 2 Loop widening is performed before and after loop bodies via `sema_widen_loop`, which sets the range of every variable assigned inside the loop to `range_unknown()`. This models the conservative fixpoint used by the VRA.

9.6 Mutual recursion detection

- 1 After all functions are processed, `sema_check_no_mutual_recursion(decls)` ([src/sema.h:786](#)) performs DFS over the static call graph.
- 2 For each non-generic `DECL_FUNCTION` (`func`, not `proc`), a depth-first traversal follows `EXPR_CALL` edges. When the DFS revisits a node already on the current stack, a cycle is detected and error `[E011]` is emitted.
- 3 The DFS is bounded by `MREC_MAX_NODES = 256`. Functions beyond this limit are silently skipped; this is acceptable for the current compiler which does not target programs with more than 256 pure functions in a call cycle. `proc` declarations are excluded from cycle detection because procedures are not required to terminate.

Note

Generic (`comptime`) functions are excluded from cycle detection because each instantiation is a distinct function body. Cycles through instantiations cannot be checked without tracking per-instantiation call graphs.

Chapter 10

Name Resolution

- 1 Name resolution is implemented in [src/sema/resolve.h](#). It is Phase 1 of semantic analysis (section 9.1): it visits every expression and statement in a function body, populates `->decl` pointers on identifier nodes, and performs a set of source-to-source transforms before type inference and checking.

10.1 Identifier resolution

- 1 `sema_resolve_expr` handles `EXPR_IDENTIFIER` by:
 1. Calling `sema_lookup(name)` to search locals then globals.
 2. If found: setting `e->decl = sym->decl`, `e->type = sym->type`, and `e->is_global = sym->is_global`.
 3. If not found: emitting an “undefined identifier” diagnostic and exiting.

10.2 UFCS desugaring

- 1 Uniform Function Call Syntax (UFCS) allows method-style calls to be written as `receiver.func(args)`. The resolver detects `EXPR_CALL` whose callee is `EXPR_MEMBER` and the member name resolves to a top-level function. It desugars the call by prepending the receiver as the first argument:

Algorithm

UFCS desugaring for `recv.f(a, b)`:

1. Parse: `CALL(MEMBER(recv, "f"), [a, b])`
2. Lookup "f" in globals; if it resolves to a `DECL_FUNCTION` or `DECL_PROCEDURE`:
3. Rewrite to: `CALL(IDENT("f"), [recv, a, b])`

- 2 If the member name does not resolve to a global function, the expression is treated as a struct field access and left as `EXPR_MEMBER`.

10.3 Destructuring parameter expansion

- 1 When a function parameter carries a `DECL_DESTRUCT` node (syntax: `func f({x, y} Point)`), the resolver:
 1. Inserts a hidden parameter `_param_N` with the struct type.
 2. Resolves the struct type by scanning `sema_decls`.

3. For each destructured name, locates the corresponding field type and inserts a local binding mapping the name to `_param_N.field`.
- 2 At emit time, the generated C function receives the hidden parameter, and each destructured name is emitted as a field access on that hidden parameter.

10.4 Target platform constants

- 1 `src/sema/resolve.h` defines compile-time platform constants used to evaluate `@os` and `@arch` builtin expressions:

Constant	Value	Condition
LAIN_TARGET_OS	LAIN_OS_LINUX	<code>__linux__</code> defined
LAIN_TARGET_OS	LAIN_OS_WINDOWS	<code>_WIN32</code> defined
LAIN_TARGET_OS	LAIN_OS_MACOS	<code>__APPLE__</code> defined
LAIN_TARGET_ARCH	LAIN_ARCH_X86_64	<code>__x86_64__</code> defined
LAIN_TARGET_ARCH	LAIN_ARCH_AARCH64	<code>__aarch64__</code> defined

- 2 These constants are compiled into the Lain compiler binary and reflect the host platform. Cross-compilation would require passing them as flags; this is not currently supported.

10.5 Scope building

- 1 `sema_build_scope(decls, module_path)` populates `sema_globals` with all top-level declarations before Phase 1 begins. It iterates over the flat `DeclList` and calls `sema_insert_global` for each `DECL_FUNCTION`, `DECL_PROCEDURE`, `DECL_STRUCT`, `DECL_ENUM`, and `DECL_TYPE_ALIAS`.
- 2 Global scope is built once per module before any function body is resolved. This allows forward references: a function `f` can call a function `g` that appears later in the source file.

Chapter 11

Type Inference

- 1 Type inference is implemented in [src/sema/typecheck.h](#). It runs during Phase 2 (`walk_stmt`) via `sema_infer_expr(e)`, which visits an expression and sets `e->type`.

11.1 Builtin type singletons

- 1 Two singleton constructors create canonical type nodes for built-in types:
 - `get_builtin_int_type()` — returns a lazily-created `Type*` for `int` with `base_type->name = "int"`.
 - `get_builtin_u8_type()` — returns the canonical `u8` type.
- 2 These singletons are stored in `static` local variables. Because all compiler-internal type construction happens in `sema_arena`, the singletons are created on first use and reused thereafter without re-allocation.

11.2 Integer widening

- 1 Lain allows implicit widening of integer types in arithmetic expressions. The widening rules are implemented by four helpers in [src/sema/typecheck.h:58](#):

Helper	Function
<code>is_integer_type(t)</code>	True for <code>u8/i8/u16/i16/u32/i32/u64/i64/int/usize/iseize</code>
<code>integer_rank(t)</code>	Rank 1–5 (1=8-bit, 2=16-bit, 3=32-bit/int, 4=64-bit, 5=usize/iseize)
<code>wider_integer_type(a,b)</code>	Returns the type with the higher rank
<code>can_widen_to(from,to)</code>	True if <code>from</code> rank $\leq$ <code>to</code> rank

- 2 For binary arithmetic and bitwise operators (`+`, `-`, `*`, `/`, `%`, `&`, `|`, `^`, `<<`, `>>`), type inference returns the wider of the two operand types. Comparison and logical operators always return `int`. Narrowing (e.g. `u64` to `u8`) still requires an explicit `as` cast. Float $\leftrightarrow$ integer conversion also requires explicit `as`.

11.3 Type compatibility

- 1 `types_compatible(from, to)` ([src/sema/typecheck.h:98](#)) is a conservative structural compatibility check used to diagnose argument/field type mismatches. It returns `true` for:
 - `NULL` on either side (missing information; skip).

- Same simple type name.
- Integer widening (from rank $\leq$ to rank).
- Pointer-to-same-element-type.

It returns `false` for clear mismatches (`int` vs `bool`, struct A vs struct B).

11.4 Inference for each ExprKind

1 `sema_infer_expr(e)` dispatches on `e->kind`:

Kind	Type assigned
EXPR_LITERAL	<code>int</code>
EXPR_FLOAT_LITERAL	<code>f64</code>
EXPR_STRING	<code>u8[:0]</code> (NUL-sentinel slice)
EXPR_CHAR	<code>u8</code>
EXPR_BINARY	Wider operand type (arithmetic) or <code>int</code> (comparison/logic)
EXPR_UNARY	Operand type (or <code>int</code> for logical not)
EXPR_IDENTIFIER	Type from symbol table
EXPR_CALL	Return type of callee declaration
EXPR_MEMBER	Type of the named field in the struct declaration
EXPR_INDEX	Element type of the array/slice
EXPR_CAST	Target type of the cast
EXPR_MOVE	Inner expression type with <code>MODE_OWNED</code>
EXPR_MUT	Inner expression type with <code>MODE_MUTABLE</code>
EXPR_ARRAY_LITERAL	Array type inferred from elements
EXPR_RANGE	Range type (<code>int..int</code>)

11.5 String type inference and generics

- 1 String literals produce `u8[:0]`, the NUL-sentinel slice type. When a string is passed to a comptime-generic function parameter that expects `comptime u8[:]`, the type is unified by the generic instantiation pass (chapter 18) which creates a specialised `Fixed_u8_N` type for fixed-length strings. The type inference layer emits `Slice_u8_0` for all runtime string values.

11.6 Parameter constraint propagation

- 1 After parameter symbols are inserted, the orchestrator applies inline parameter constraints (e.g. `n int >= 0`) and pre-contract expressions to the `RangeTable`. `sema_apply_constraint(expr, ranges)` interprets a binary comparison expression as a range restriction on the named variable and calls `range_set` accordingly. This seeds VRA with the proven parameter bounds before any statement in the body is analysed.

Chapter 12

Ownership and Linearity

- 1 The linearity checker enforces Lain’s ownership model: every value of a linear type must be consumed exactly once. It is implemented in `src/sema/linearity.h` and runs after both sema phases via `sema_check_function_linearity(d)`.

12.1 Linear type classification

- 1 A type is linear if:
 1. Its `OwnershipMode` is `MODE_OWNED` (set by `mov T`), or
 2. It is a fixed-length array (`TYPE_ARRAY`) whose element type is linear, or
 3. It is a `TYPE_SIMPLE` struct or enum that contains at least one linear field (computed recursively).
- 2 Slices, pointers, and dynamic arrays are *not* linear by default: they are views, not owners. Only `TYPE_ARRAY` (which owns its elements) propagates linearity from element to container.

12.2 Linearity table

- 1 The checker maintains an `LTable` — a linked list of `LEntry` nodes, one per linear variable in the current function:

LEntry `src/sema/linearity.h:120`

Field	Type	Semantics
<code>id</code>	<code>Id *</code>	Variable identifier.
<code>state</code>	<code>LState</code>	<code>LSTATE_UNCONSUMED</code> or <code>LSTATE_CONSUMED</code> .
<code>must_consume</code>	<code>bool</code>	True if type is strictly linear.
<code>is_initialized</code>	<code>bool</code>	True if definitely initialized.
<code>is_defer_consumed</code>	<code>bool</code>	True if consumed inside a <code>defer</code> block.
<code>field_states</code>	<code>FieldState *</code>	Per-field tracking for struct types (see below).
<code>defined_loop_depth</code>	<code>int</code>	Loop nesting depth at declaration.
<code>region</code>	<code>Region *</code>	Borrow region at declaration.
<code>line, col</code>	<code>isize</code>	Source location for diagnostics.

- 2 The four `LState` values are:

<code>LState</code>	Meaning
<code>LSTATE_UNCONSUMED</code>	Variable is live and available.
<code>LSTATE_CONSUMED</code>	Variable has been consumed (via <code>mov</code> or at last use).
<code>LSTATE_BORROWED_READ</code>	Reserved for future read-borrow state.
<code>LSTATE_BORROWED_WRITE</code>	Reserved for future write-borrow state.

12.3 Field-level tracking

- 1 When a struct variable has multiple linear fields, the checker tracks each field independently via a linked list of `FieldState` nodes. A field is consumed when a `mov` expression targets it: `mov s.field`.
- 2 When all linear fields of a struct are consumed, the whole-variable entry is automatically promoted to `LSTATE_CONSUMED`. This allows partial consumption patterns where individual fields are moved out of a struct without requiring a whole-struct move.

12.4 Consumption rules

- 1 The checker enforces the following rules:

- [E001] *Use after move.* An identifier expression in a non-move context on a variable with `LSTATE_CONSUMED` is an error.
- [E002] *Double move.* A `mov` expression on a variable already in `LSTATE_CONSUMED` is an error.
- [E003] *Unconsumed linear value.* After a function body completes, any `must_consume` variable still in `LSTATE_UNCONSUMED` is an error.
- [E006] *Move inside loop.* Consuming a linear variable that was declared outside the current loop nesting level is an error; the variable could be consumed on the first iteration and accessed on the second.

12.5 Borrow interaction

- 1 The `LTable` carries a `BorrowTable *borrows` pointer. Before a variable can be moved (`mov x`), the checker verifies that no active borrow of `x` exists in the borrow table. This integrates ownership checking with the region-based borrow checker described in chapter 13.

12.6 Non-Lexical Lifetimes (NLL)

- 1 `src/sema/use_analysis.h` implements a use-analysis pre-pass that identifies the *last use* of each linear variable. A “last use” of a linear variable that appears as a function argument may be treated as an implicit move, eliminating the need for an explicit `mov` at every call site. The pre-pass annotates `EXPR_CALL` argument expressions so the linearity checker can treat them as consuming moves.

Chapter 13

Borrow Checking and Regions

- 1 The borrow checker ensures that references do not outlive their owners and that the aliasing invariant (one mutable borrow *or* many shared borrows) is maintained. It is implemented in [src/sema/region.h](#) and is invoked from the linearity checker ([src/sema/linearity.h](#)).

13.1 Regions

- 1 A **Region** represents a lexical scope boundary. Regions are organized in a parent chain from inner to outer:

Region [src/sema/region.h:38](#)

Field	Type	Semantics
id	int	Unique identifier (monotonically increasing).
depth	int	Nesting depth (0 = function scope).
parent	Region *	Enclosing region (NULL for the outermost scope).

- 2 `region_contains(outer, inner)` walks the parent chain of `inner` to check whether `outer` appears in it. If so, `inner` is a sub-scope of `outer` and will terminate before `outer`.
- 3 The invariant checked by the borrow checker is: the borrow region must be contained within the owner region. That is, `region_contains(owner_region, borrow_region)` must hold, guaranteeing the owner outlives the borrow.

13.2 Borrow entries

- 1 An active borrow is recorded as a **BorrowEntry**:

BorrowEntry [src/sema/region.h:78](#)

Field	Type	Semantics
var	Id *	The borrowed variable.
owner_var	Id *	Direct owner of the borrow.
root_owner	Id *	Ultimate non-reference owner (for transitive chains).
mode	OwnershipMode	MODE_SHARED or MODE_MUTABLE.
phase	BorrowPhase	BORROW_RESERVED or BORROW_ACTIVE.
borrow_region	Region *	Scope where the borrow is used.
owner_region	Region *	Scope where the owner is defined.
is_temporary	bool	Expires at end of current statement.
last_use_stmt_idx	int	NLL: last statement index where binding is used.

13.3 Two-phase borrows

- 1 Lain implements two-phase borrows to permit calling a method on a container while simultaneously passing container elements as arguments. A borrow begins in the `BORROW_RESERVED` phase, which allows shared reads but does not yet activate the exclusivity constraint. After all call arguments have been evaluated, the borrow is promoted to `BORROW_ACTIVE` via `borrow_promote_to_active()`.
- 2 The promotion sequence in the expression handler for `EXPR_CALL`:
 1. Register a reserved borrow for the implicit receiver argument.
 2. Evaluate all call arguments (with the borrow in `BORROW_RESERVED`).
 3. Call `borrow_promote_to_active()` to activate the borrow.
- 3 This follows the model described in *Two-Phase Borrows* (Matsakis 2018). The key insight is that a reserved borrow does not conflict with shared reads of the same variable, so the argument evaluation (which may read the container) is allowed before the borrow becomes exclusive.

13.4 Aliasing rules

- 1 The `BorrowTable` enforces the following aliasing rules at every borrow registration:
 1. **No mutable borrow while any borrow is active.** If the requested mode is `MODE_MUTABLE` and any active borrow of the same root owner exists, the operation is rejected.
 2. **No shared borrow while a mutable borrow is active.** If the requested mode is `MODE_SHARED` and an active mutable borrow of the same root owner exists, the operation is rejected.
- 2 Temporary borrows (`is_temporary = true`) are released at the end of each statement. Non-temporary borrows are released when their binding goes out of scope.

13.5 NLL last-use optimization

- 1 The `last_use_stmt_idx` field supports Non-Lexical Lifetime (NLL) shortening. The use-analysis pre-pass (`src/sema/use_analysis.h`) assigns to each borrow the index of the last top-level statement that uses the binding. The borrow checker can then expire a borrow as soon as it is no longer used, even before the lexical scope boundary is reached. This allows code like:

```
1 var x = acquire_resource()
2 use x as r
3 // r's borrow expires here (last use above)
4 // x can now be moved below:
5 f(mov x)
```


Chapter 14

Value Range Analysis

- 1 Value Range Analysis (VRA) tracks compile-time integer value ranges for local variables throughout a function body. It is the core mechanism by which the compiler statically proves array accesses safe without runtime checks. VRA is implemented in [src/sema/ranges.h](#).

14.1 Range representation

- 1 A range is a closed integer interval $[\text{min}, \text{max}]$:

```
1 typedef struct {
2     int64_t min;
3     int64_t max;
4     bool    known;
5 } Range;
```

- 2 `known = false` means the range is unknown (unbounded). Arithmetic on an unknown range propagates the unknown: any operation whose operand is unknown returns `range_unknown()`. Three constructors:

Constructor	Produces
<code>range_unknown()</code>	Unknown (<code>known=false</code> , <code>min=max=0</code>)
<code>range_const(v)</code>	$[v, v]$ — a known singleton
<code>range_make(lo, hi)</code>	$[lo, hi]$

14.2 Saturating arithmetic

- 1 All range arithmetic uses saturating 64-bit operations to prevent silent integer overflow. Four helpers:

Helper	Saturates at
<code>sat_add_i64(a,b)</code>	<code>INT64_MAX/INT64_MIN</code>
<code>sat_sub_i64(a,b)</code>	<code>INT64_MAX/INT64_MIN</code>
<code>sat_mul_i64(a,b)</code>	<code>INT64_MAX/INT64_MIN</code> , handles <code>INT64_MIN/-1</code> UB
<code>sat_div_i64(a,b)</code>	guards <code>INT64_MIN/-1</code> UB, returns 0 for <code>b=0</code>

14.3 RangeTable

- 1 A `RangeTable` maps variable `Id *` to `Range`. It also maintains a list of `ConstraintEntry` records for linear constraints of the form $x = y + c$:

- `range_set(table, id, range)` inserts or updates the range for a variable.
- `range_get(table, id)` looks up a variable's range; returns `range_unknown()` if not found.
- `sema_eval_range(expr, table)` recursively evaluates an expression to a `Range` by combining sub-expression ranges with saturating arithmetic.

14.4 Constraint propagation

- 1 The `STMT_ASSIGN` handler in `walk_stmt` adds linear constraints to the table when the right-hand side has the form `y + c`, `c + y`, or `y - c`:

```

1 // x = y + c
2 constraint_add(table, x_id, y_id, c);
3 constraint_add(table, y_id, x_id, -c);

```

- 2 These bidirectional constraints allow the VRA to tighten bounds on either variable when the other is constrained. For example, if `y` is known in range `[0, 7]` and `x = y + 1`, the VRA can infer `x ∈ [1, 8]`.

14.5 Condition-driven range refinement

- 1 `sema_apply_constraint(cond, table)` interprets a boolean condition as a range restriction. For example, `if x < 10` causes the VRA to tighten the upper bound of `x` to 9 in the then-branch. `sema_apply_negated_constraint` applies the negation in the else-branch.
- 2 Loop widening is applied before and after every loop body. `sema_widen_loop(body, table)` sets the range of any variable assigned inside the loop to `range_unknown()`. This is a conservative over-approximation of the fixpoint; the VRA does not compute widening operators.

14.6 In-guard table

- 1 The `in` operator (e.g. `idx in arr`) serves as a bounds-proving condition. When used as a loop condition or if-condition, it implies $0 \leq \text{idx} < \text{arr.len}$. The in-guard table records these proven guards so the bounds checker can skip runtime checks for subscripts inside the guarded region.

`InGuardEntry` [src/sema.h:43](#)

Field	Type	Semantics
<code>index</code>	<code>Expr *</code>	The index expression proven in-bounds.
<code>container</code>	<code>Expr *</code>	The array/slice expression.
<code>next</code>	<code>InGuardEntry *</code>	Next entry in the linked list.

- 2 `sema_push_in_guards(cond)` extracts every `in` sub-expression from a (possibly conjunctive) condition and adds an entry. Guards are pushed at the start of an `if` or `while` body and popped on scope exit. `sema_is_in_guarded(index, container)` performs a structural-equality search through the guard list.

Chapter 15

Static Bounds Checking

- 1 The bounds checker verifies that every array and slice subscript is within the valid range $[0, \text{length} - 1]$ before allowing the expression to reach the code generator. It is implemented in `src/sema/bounds.h` and called from the type-checking walk (`sema_infer_expr`) when processing `EXPR_INDEX` nodes.

15.1 Check algorithm

- 1 For each `EXPR_INDEX` expression (syntax: `arr[idx]`):

Algorithm

Bounds check for `arr[idx]`:

1. If `sema_in_unsafe_block` is true: skip all checks (unsafe block explicitly disables bounds verification).
2. If `sema_walk_phase` is false: skip (bounds checking only runs during walk phase).
3. If `idx` is an `EXPR_RANGE` (`arr[lo..hi]`): check both endpoints independently.
4. **In-guard check:** if `sema_is_in_guarded(idx, arr)` returns true, the index is proven in-bounds by a live `in` guard; skip.
5. **VRA check:** evaluate `sema_eval_range(idx, sema_ranges)`. If the resulting range $[min, max]$ satisfies $0 \leq min$ and $max < arr.len$ (for a statically known length), the access is proven safe; skip.
6. **Runtime check:** if neither static proof applies and the array length is known, emit a runtime bounds check in the generated C output.
7. If the VRA proves the access is *definitely out of bounds* (i.e. $min \geq arr.len$ or $max < 0$), emit a compile-time error **[E045]** (or an appropriate bounds error code).

15.2 Safe fragment guarantee

- 1 In the safe fragment (no `unsafe` blocks), the compiler guarantees that all array accesses are either statically proven in-bounds or guarded by a runtime check in the generated C code. This guarantee is contingent on:

- The VRA not being circumvented by pointer arithmetic.
- The in-guard table being correctly maintained (no double-pop errors).

- The `sema_walk_phase` flag being set correctly during walk.

Invariant

No `EXPR_INDEX` node that reaches `emit_expr()` has been subjected to bounds checking in `unsafe` mode without explicit opt-in. Every subscript either has a static proof or a generated runtime check.

15.3 Interaction with VRA

- 1 The bounds checker cooperates with VRA in two directions:
 1. **VRA proves bounds:** if VRA has refined the range of the index to be within $[0, len-1]$, no runtime check is emitted.
 2. **Constraint seeding:** `in` conditions (`idx in arr`) seed the VRA with the constraint $0 \leq idx < arr.len$, which is then available as a proven fact for all code inside the guard.

15.4 Interaction with unsafe

- 1 Inside `unsafe { ... }` blocks, `sema_in_unsafe_block` is `true` and bounds checking is skipped entirely. The emitter generates raw unchecked array subscripts. The programmer is responsible for any out-of-bounds accesses that may result. This is the intended escape hatch for performance-critical code that has been verified by other means.

Chapter 16

Pattern Matching Exhaustiveness

- 1 The exhaustiveness checker verifies that every `case` expression or statement covers all possible values of the scrutinee type. It is implemented in `src/sema/exhaustiveness.h` and called from the resolve pass for every `STMT_MATCH` and `EXPR_MATCH` node.

16.1 Wildcard arm

- 1 A match that contains an `else` arm (the wildcard case) is always exhaustive. The wildcard arm has `patterns = NULL` in the AST. The checker returns immediately upon encountering such an arm.

16.2 Enum exhaustiveness

- 1 For enum scrutinees, the checker enumerates all variants in the enum declaration and verifies that each variant name appears as a pattern in at least one arm. A variant is considered covered if:
 - An arm has a pattern that is an `EXPR_IDENTIFIER` whose name matches the variant name, or
 - An arm has a pattern that is an `EXPR_CALL` (constructor pattern) whose callee name matches the variant name.
- 2 If any variant is not covered and there is no wildcard arm, the checker emits an exhaustiveness error listing the uncovered variants and exits.

16.3 Boolean exhaustiveness

- 1 For `bool` scrutinees, the checker verifies that both `true` and `false` appear as patterns. A match on `bool` with only one branch and no wildcard is rejected.

16.4 Integer and other types

- 1 For integer scrutinees, exhaustiveness cannot generally be proven without a wildcard arm (because the integer range is too large to enumerate). The checker requires a wildcard (`else`) arm for any match on an integer type.
- 2 For struct types, the checker does not attempt exhaustiveness; a struct is a single value and any arm that matches a struct literal is sufficient. A wildcard arm is still required if the match is not intended to cover every possible struct layout.

Chapter 17

Termination Verification

- 1 Lain's `func` declarations are required to terminate. Termination is guaranteed by the combination of three rules:
 1. Direct recursion is rejected in `func` at the resolve phase.
 2. Mutual recursion among `func` declarations is rejected by `sema_check_no_mutual_recursion` (section 9.6).
 3. Unbounded `while` loops are forbidden in `func`. A `while` loop with a `decreasing` measure is allowed because the compiler verifies termination.
- 2 The bounded-while verifier is self-contained in `src/sema.h:344` and does *not* use the VRA range table, because VRA cannot track measures that involve struct fields.

17.1 Termination measure requirements

- 1 A valid termination measure must satisfy two conditions:
 1. **Non-negativity**: when the loop condition holds, the measure is ≥ 0 .
 2. **Strict decrease**: the loop body strictly decreases the measure.
- 2 The verifier checks these conditions syntactically and raises errors if they cannot be proven:

Error	Condition	Hint
[E080]	Non-negativity cannot be proven	Use <code>while a < b decreasing b - a</code>
[E081]	No variables in measure	Measure must reference identifiers/fields
[E082]	No strict decrease found	Body must contain an assignment that decreases the measure

17.2 Non-negativity verification

- 1 `sema_verify_measure_nonneg(cond, measure)` checks whether the loop condition syntactically implies the measure is non-negative. It recognizes two patterns:
 1. **Difference pattern**: measure has the form `hi - lo` and the condition contains a comparison `lo < hi` (strict) or `lo <= hi`. Example: `while a < b decreasing b - a`.
 2. **Single-variable pattern**: measure is a single identifier `n` and the condition contains `n > 0` or `n > constant >= 0`. Example: `while n > 0 decreasing n`.

3. **In-guard pattern:** condition is `idx in arr` and measure is `arr.len - idx`. The `in` guard implies $0 \leq \text{idx} < \text{arr.len}$, so the measure is $\geq 1 > 0$.
- 2 Conjunctive conditions (`a and b`) are traversed recursively; either conjunct may supply the non-negativity proof.

17.3 Strict decrease verification

- 1 `measure_scan_body(body, vars, nvar)` scans the loop body for assignments to variables that appear in the measure expression. `measure_extract_vars` first decomposes the measure into (variable, polarity) pairs:
 - Measure `x`: variable `x` with polarity `+1`.
 - Measure `a - b`: variable `a` with polarity `+1`, variable `b` with polarity `-1`.
- 2 For each assignment `v = v op k` (where `v` is a measure variable and `k` is a literal), `assignment_direction` determines whether the assignment increases or decreases `v`. The effect on the measure is `polarity * direction`:
 - Negative effect (direction opposes polarity): the measure decreases; record `found_decrease = true`.
 - Positive effect (direction reinforces polarity): the measure *increases*; immediately return conflict `(-1)`.
- 3 If no assignment to any measure variable is found (`found_decrease` remains false), the verifier returns 0 and error `[E082]` is emitted.

17.4 Accepted measure patterns

- 1 Practical examples of accepted termination measures:

```

1 // while n > 0 decreasing n { n -= 1 }
2 while n > 0 decreasing n {
3     n -= 1
4 }
5
6 // while pos < size decreasing size - pos { pos += 1 }
7 while pos < size decreasing size - pos {
8     pos += 1
9 }
10
11 // while idx in arr decreasing arr.len - idx { idx += 1 }
12 while idx in arr decreasing arr.len - idx {
13     idx += 1
14 }

```

- 2 All three patterns are accepted because: (1) the condition implies the measure is positive, and (2) the body contains an assignment that decreases the measure by a positive literal amount.

Part V

Code Generation

Chapter 18

Compile-Time Evaluation and Generics

- 1 Lain’s compile-time evaluation (CTFE) system evaluates a subset of the AST during compilation to support generic functions, type aliases, and comptime-if branches. The CTFE engine is in `src/sema/comptime.h`; generic instantiation is in `src/sema/generic.h`.

18.1 Comptime environment

- 1 The CTFE interpreter maintains a linked list of bindings:

```
1 typedef struct ComptimeEnv {  
2     Id      *name;  
3     Expr    *value;  
4     struct ComptimeEnv *next;  
5 } ComptimeEnv;
```

- 2 `comptime_env_push` prepends a new binding to the front of the list. `comptime_env_lookup` performs a linear search by name. The environment is passed by value through the recursive evaluator; there is no heap-allocated scope stack.

18.2 Expression evaluation

- 1 `comptime_evaluate_expr(arena, expr, env)` reduces an expression to a value. It handles:
 - `EXPR_LITERAL` and `EXPR_STRING`: returned as-is.
 - `EXPR_IDENTIFIER`: looked up in `env`, then in globals. Struct/enum/type names are wrapped in an `EXPR_TYPE` node. Primitive type names (`int`, `u8`, etc.) are returned as canonical `EXPR_TYPE` nodes.
 - `EXPR_BINARY`: arithmetic operators are evaluated numerically and returned as `EXPR_LITERAL`.
 - `EXPR_CALL`: dispatched to `comptime_evaluate_function`.
- 2 Recursion depth is bounded by `COMPTIME_MAX_DEPTH = 256` to prevent stack overflow from runaway CTFE loops. Exceeding this limit exits with error `[E045]`.

18.3 Generic function instantiation

- 1 A function with at least one `comptime` parameter is a generic function. When a call to such a function is resolved, the resolver:

1. Evaluates all comptime arguments at compile time.
 2. Deep-clones the function body using `src/ast_clone.h`.
 3. Calls `generic_substitute_expr` (from `src/sema/generic.h`) to replace occurrences of the comptime parameter names with the evaluated types or values.
 4. Registers the specialised clone as a new `DECL_FUNCTION` with a mangled name and inserts it into the global scope.
 5. Resolves the specialised body recursively.
- 2 Because specialisation happens during name resolution, every generic function appears in the emitted C output as one or more monomorphic functions with mangled names derived from the argument types. There is no runtime polymorphism.

18.4 Comptime-if

- 1 A `STMT_COMPTIME_IF` evaluates its condition at compile time via `comptime_evaluate_expr`. The condition typically tests `@os` or `@arch` builtin values against platform constants. The resolver records which branch was selected in `StmtComptimeIf.is_taken`; the walk phase only analyses the selected branch. The unselected branch is never type-checked or emitted.

18.5 Type alias evaluation

- 1 `type` `Name` = `Expr` declarations are evaluated at resolve time. `comptime_evaluate_function` is called with the right-hand side expression and the current environment. For parameterised aliases (e.g. `type Option(comptime T) = ...`), the argument is substituted before evaluation. The result is stored in the `DeclTypeAlias` node and used by the emitter to generate the corresponding C `typedef`.

Chapter 19

Code Generation

- 1 The code generator translates the annotated AST into valid C99 source code. It is split across `src/emit/core.h`, `src/emit/decl.h`, `src/emit/stmt.h`, `src/emit/expr.h`, `src/emit/ctor.h`, `src/emit/type_order.h` and `src/emit/lain_header.h`.

19.1 Emit context

- 1 All output is written through a single macro:

```
1 static FILE *output_file;  
2 #define EMIT(...) fprintf(output_file, __VA_ARGS__)
```

- 2 Indentation is managed by `emit_indent(depth)`, which writes `4 * depth` spaces. The depth is a plain integer passed through recursive emit functions.

19.2 Output structure

- 1 The generated C file has the following layout:
 1. **Runtime header:** the content of `src/emit/lain_header.h` is emitted verbatim. It defines macro aliases for C library functions (`libc_printf`, `libc_puts`, etc.) and provides the slice type primitives.
 2. **c_include directives:** any `c_include "foo.h"` declarations are emitted as `#include "foo.h"`.
 3. **Type forward declarations:** struct and enum types are forward-declared via `emit/type_order.h` in topological dependency order, preventing use-before-definition errors in C.
 4. **Struct/enum definitions:** full `typedef struct` and `typedef enum` bodies.
 5. **Function prototypes:** all functions are forward-declared to allow mutual calls.
 6. **Function bodies:** each `DECL_FUNCTION` and `DECL_PROCEDURE` is emitted with its full body.

19.3 Type emission

- 1 `c_name_for_type(t, out, cap)` converts a Lain Type `*t` to a C type string. The mapping:

Lain type	C output
<code>int</code>	<code>int</code>
<code>u8</code>	<code>uint8_t</code>
<code>i64</code>	<code>int64_t</code>
<code>bool</code>	<code>bool</code>
<code>f32</code>	<code>float</code>
<code>f64</code>	<code>double</code>
<code>u8[5]</code>	<code>uint8_t[5]</code>
<code>u8[:0]</code>	<code>Slice_u8_0</code> (generated typedef)
<code>mov T</code>	Same as T (ownership is erased)
<code>mut T</code>	Same as T * (mutable borrow becomes pointer)

- Slice types with sentinels (e.g. `u8[:0]`) are represented as C structs with a `data` pointer and a `len` field. The first use of a slice type triggers `emit_slice_type_definition` to emit the typedef, which is then reused for all subsequent uses of the same type.

19.4 Enum emission

- Lain enums with payload variants emit:
 - A `typedef enum {...}` for the tag type.
 - A `typedef struct {...}` with a `tag` field and a union of variant payloads.
- Constructor functions are generated by `src/emit/ctor.h` for each variant. For a variant `Some(value int)`, the constructor is:

```
Option Option_Some(int value) {
    Option result;
    result.tag = Option_Some_tag;
    result.Some.value = value;
    return result;
}
```

19.5 Defer mechanics

- `defer` statements are implemented using a static stack of up to `MAX_DEFERS = 256` statements. On every return path, the emitter iterates the defer stack in reverse order and emits each deferred statement before the `return`. Loop-level defer tracking ensures deferred statements run at loop exit as well as at function exit.

19.6 String literal initializers

- String literals initializing typed variables are handled by `emit_fixed_string_init`. For a `u8[:0]` target, it emits a compound literal:

```
(Slice_u8_0){ .len = 5, .data = (uint8_t[]){ 'h','e','l','l','o',
    0 } }
```

- The sentinel byte is always appended explicitly. This avoids reliance on C's implicit NUL-termination of string literals, which may not match the sentinel value for non-zero sentinels.

Part VI

Standard Library and Testing

Chapter 20

Standard Library Implementation

- 1 The Lain standard library consists of a set of `.ln` source files under `std/`. These files are written in Lain itself and are imported by user programs via `import std.io`, `import std.math`, etc. The standard library is compiled along with user code in the same single-pass pipeline; there is no pre-compiled library archive.

20.1 `std.c` — C interoperability

- 1 `std/c.ln` declares the C standard library interface using `extern proc` and `extern func` declarations. These declarations make C functions available to Lain code without requiring a `c_include` directive in user code. The module also declares the opaque `FILE` type via `extern type FILE`.

20.2 `std.io` — Console I/O

- 1 `std/io.ln` provides basic console output. It wraps the C `printf` and `puts` functions:

Function	Description
<code>print(s u8[:0])</code>	Print a string without newline.
<code>println(s u8[:0])</code>	Print a string followed by a newline.

20.3 `std.fs` — File system

- 1 `std/fs.ln` provides basic file I/O. The `File` type is linear (owned): it must be explicitly closed after use.

Function	Signature	Description
<code>open</code>	<code>proc(path u8[:0])mov File</code>	Open file for writing.
<code>close</code>	<code>proc(f mov File)</code>	Close and consume the file handle.
<code>write</code>	<code>proc(f mut File, s u8[:0])</code>	Write a string to a file.

- 2 The `File` type is declared as `extern type` backed by a C `FILE *`. Opening returns a `mov File` (owned), ensuring the caller cannot forget to close it without a compiler error.

20.4 `std.math` — Integer math

- 1 `std/math.ln` provides pure integer math utilities implemented directly in Lain:

Function	Description
<code>min(a int, b int)int</code>	Minimum of two integers.
<code>max(a int, b int)int</code>	Maximum of two integers.
<code>abs(x int)int</code>	Absolute value.
<code>clamp(x int, lo int, hi int)int</code>	Clamp x to $[lo, hi]$.

- 2 All four functions are declared `func` (pure): they contain no loops and return immediately, satisfying the termination guarantee.

20.5 std.string — String utilities

- 1 `std/string.ln` provides string operations over `u8[:0]` slices (NUL-sentinel strings). Functions include length computation, substring extraction, and comparison.

20.6 std.option and std.result

- 1 `std/option.ln` and `std/result.ln` define generic algebraic data types using comptime generics:

```
1 enum Option(comptime T type) {  
2     Some { value T }  
3     None  
4 }  
5  
6 enum Result(comptime T type, comptime E type) {  
7     Ok { value T }  
8     Err { error E }  
9 }
```

- 2 These are the standard vocabulary types for optional and fallible computations. Instantiation is monomorphic: each distinct type argument combination produces a separate C struct in the output.

Chapter 21

Testing Infrastructure

- 1 The Lain compiler's correctness is verified by a test suite of Lain source files under `tests/`. Tests are driven by the shell script `run_tests.sh`.

21.1 Test file conventions

- 1 Test files follow a naming convention that determines the expected outcome:

Name pattern	Expected outcome
<code>*_pass.ln</code>	Compilation must succeed (exit code 0).
<code>*_fail.ln</code>	Compilation must fail (exit code non-zero).
Other <code>.ln</code>	Treated as pass tests by default.

- 2 A `_fail.ln` test may additionally contain a comment of the form `// EXPECT: [EXXX]` in the source text. If present, the test framework verifies that the compiler's `stderr` output contains the expected diagnostic code. This allows negative tests to verify not just that the compiler rejects the program but that it emits the *correct* error.

21.2 Test runner

- 1 `run_tests.sh` (`run_tests.sh`):
 1. Builds the compiler if not already present.
 2. Recursively discovers all `.ln` files under `tests/`.
 3. For each file, runs the compiler and compares the exit code against the expected outcome.
 4. For `_fail.ln` files with `// EXPECT:` comments, also matches the diagnostic code against `stderr`.
 5. Reports pass/fail counts and lists all failed tests.

21.3 Test categories

- 1 Tests are organized into subdirectories under `tests/`:

Directory	Tests
tests/safety/	Ownership, linearity, borrow, and unsafe tests
tests/safety/bounds/	Bounds checking (pass and fail)
tests/safety/purity/	Purity / termination verification
tests/types/	Type inference, widening, mismatch detection
tests/emit/	Code generation correctness
tests/stdlib/	Standard library function tests
tests/generics.ln	Comptime generic instantiation
tests/core/	Core language features
tests/borrowck/	Borrow checking pass and fail tests

21.4 Invoking the test suite

- 1 To run the full test suite from the repository root:

```
1 bash run_tests.sh
```

- 2 The script builds the compiler automatically if the `lain` binary is not present. Exit code 0 means all tests passed; non-zero means at least one test failed. Failed test names are printed to `stdout`.
- 3 To run a single test manually:

```
1 ./lain tests/safety/bounds/dynamic_len_known_idx_fail.ln
2 echo "exit: $?"
```

- 4 The expected exit code for a `_fail` test is 1 (or any non-zero value). For a `_pass` test, exit code 0 is required.

AST Node Reference

- ⁵ This annex provides a complete enumeration of every AST node discriminant defined in [src/ast.h](#). For architectural context and struct field descriptions see chapter 4.

.1 ExprKind

Table 1: ExprKind enumeration.

Kind	Payload	Key fields
EXPR_BINARY	ExprBinary	left, op, right
EXPR_UNARY	ExprUnary	op, right
EXPR_IDENTIFIER	ExprIdentifier	id (Id *)
EXPR_LITERAL	ExprLiteral	value (int)
EXPR_FLOAT_LITERAL	ExprFloat	value (double)
EXPR_STRING	ExprString	value (const char *), length
EXPR_CHAR	ExprChar	value (char)
EXPR_MEMBER	ExprMember	target, member (Id *)
EXPR_CALL	ExprCall	callee, args (ExprList *)
EXPR_INDEX	ExprIndex	target, index
EXPR_RANGE	ExprRange	start, end, inclusive
EXPR_MOVE	ExprMove	expr
EXPR_MUT	ExprMut	expr
EXPR_CAST	ExprCast	expr, target_type
EXPR_MATCH	ExprMatch	value, cases, is_borrowed
EXPR_ARRAY_LITERAL	ExprArrayLiteral	elements (ExprList *)
EXPR_ANON_STRUCT	ExprAnonStruct	fields (DeclList *)
EXPR_ANON_ENUM	ExprAnonEnum	variants
EXPR_TYPE	ExprType	type_value (Type *)
EXPR_BUILTIN	ExprBuiltin	builtin_kind
EXPR_UNDEFINED	(none)	Uninitialized sentinel

- ¹ Every Expr node also carries: `->type` (set by type inference), `->decl` (set by name resolution), `->is_global`, and source position `line/col`.

.2 StmtKind

Table 2: StmtKind enumeration.

Kind	Payload	Key fields
STMT_VAR	StmtVar	name, type, expr, is_mutable
STMT_ASSIGN	StmtAssign	target, expr, is_const
STMT_EXPR	StmtExpr	expr
STMT_IF	StmtIf	cond, then_branch, else_branch
STMT_FOR	StmtFor	index_name, value_name, iterable, body
STMT_WHILE	StmtWhile	cond, measure, body
STMT_MATCH	StmtMatch	value, cases, is_borrowed
STMT_RETURN	StmtReturn	value
STMT_UNSAFE	StmtUnsafe	body
STMT_DEFER	StmtDefer	stmt
STMT_USE	StmtUse	target, alias_name
STMT_COMPTIME_IF	StmtComptimeIf	cond, then_branch, else_branch, is_taken
STMT_CONTINUE	(none)	
STMT_BREAK	(none)	

.3 DeclKind

Kind	Payload	Lain source
DECL_VARIABLE	DeclVariable	Field, parameter, local
DECL_FUNCTION	DeclFunction	func
DECL_PROCEDURE	DeclFunction	proc
DECL_EXTERN_FUNCTION	DeclFunction	extern func
DECL_EXTERN_PROCEDURE	DeclFunction	extern proc
DECL_STRUCT	DeclStruct	struct S
DECL_ENUM	DeclEnum	enum E
DECL_IMPORT	DeclImport	import
DECL_EVAL_IMPORT	DeclImport	Evaluated import
DECL_C_INCLUDE	DeclCInclude	c_include
DECL_DESTRUCT	DeclDestruct	Destructuring param
DECL_EXTERN_TYPE	DeclExternType	extern type
DECL_TYPE_ALIAS	DeclTypeAlias	type N = E

.4 TypeKind and OwnershipMode

TypeKind	Key fields	Lain syntax
TYPE_SIMPLE	base_type	<code>int</code> , <code>MyStruct</code>
TYPE_ARRAY	element_type, array_len	<code>u8[5]</code> , <code>u8[]</code>
TYPE_SLICE	element_type, sentinels	<code>u8[:0]</code>
TYPE_POINTER	element_type	<code>*T</code>
TYPE_COMPTIME	element_type	<code>comptime T</code>
TYPE_VARIANT	variant	ADT variant payload
TYPE_META_TYPE	(none)	<code>type</code> as value

OwnershipMode	Lain modifier
MODE_OWNED	<code>mov T</code>
MODE_SHARED	<i>(default, no modifier)</i>
MODE_MUTABLE	<code>mut T</code> / <code>var</code>

Diagnostic Catalogue

- 1 Every diagnostic emitted by the Lain compiler carries a bracketed code of the form [EXXX]. Each diagnostic is written to `stderr` and causes a non-zero exit code. This annex lists every code in numeric order together with the triggering condition and the source location responsible.
- 2 Diagnostics whose prefix is [VRA] are emitted by the bounds checker ([src/sema/bounds.h](#)) and do not carry a structured E-number; they are documented separately in appendix .10.

.5 Linearity and ownership errors (E001–E010)

Table 3: Linearity and ownership diagnostic codes.

Code	Summary	Triggering condition
[E001]	Use after move	An identifier whose LState is LSTATE_CONSUMED is referenced. Also raised when a borrow's root owner has been moved and the borrow is still used (src/sema/region.h).
[E002]	Double move / field re-consume	A <code>mov</code> expression is applied to an already-consumed variable, or a struct field whose FieldState is already LSTATE_CONSUMED is consumed a second time (src/sema/linearity.h).
[E003]	Unconsumed linear value	A linear variable or field reaches a <code>return</code> or the end of its scope without having been consumed. Also raised when a STMT_EXPR discards a value of a <code>mov</code> type (src/sema/linearity.h).
[E004]	Borrow aliasing violation	A new borrow conflicts with an existing live borrow: (a) a shared borrow is requested while a mutable borrow is active, (b) a mutable borrow is requested while any borrow (shared or mutable) is active. Also raised when an access is attempted on a mutably-borrowed variable (src/sema/region.h).
[E005]	Use of uninitialized variable	An identifier is accessed before any assignment has been recorded for it in the LTable (src/sema/linearity.h).

(continued)

Code	Summary	Triggering condition
[E006]	Loop-level consume	A linear variable defined outside a loop body is consumed inside the loop, or a field of such a variable is consumed inside the loop. This would cause multiple consumption across iterations (src/sema/linearity.h).
[E007]	Missing <code>mov</code> keyword	A function parameter of <code>MODE_OWNED</code> type is passed without an explicit <code>mov</code> at the call site (src/sema/linearity.h).
[E008]	Move while borrowed	A <code>mov</code> expression is applied to a variable that still has at least one active borrow, or the variable has partially-consumed fields and cannot be moved as a whole (src/sema/linearity.h).
[E009]	Assignment to immutable	An assignment target is a variable declared without <code>var</code> (i.e. <code>StmtVar.is_mutable</code> is false) (src/sema/resolve.h).
[E010]	Dangling reference	A mutable reference (<code>var T</code>) is returned from a function and the referenced variable is local to that function (lifetime analysis in both src/sema/resolve.h and src/sema/region.h).

.6 Purity and mutability errors (E011)

Table 4: Purity diagnostic codes.

Code	Summary	Triggering condition
[E011]	Purity violation	A <code>func</code> declaration violates one of three purity rules: (1) it calls a <code>proc</code> (side-effectful procedure); (2) it assigns to a global variable; (3) it participates in a mutual-recursion cycle with another <code>func</code> (checked via the DFS in src/sema.h); (4) it contains an unbounded <code>while</code> loop (one without a <code>decreasing</code> measure). All cases share the same code but produce distinct message text.

.7 Type and call errors (E012–E015)

Table 5: Type and call diagnostic codes.

Code	Summary	Triggering condition
[E012]	Type/call mismatch	A catch-all code for multiple static type errors: wrong argument count to a function; struct initializer argument count mismatch; struct field type mismatch in a positional call; arithmetic binary operator applied to a struct or enum type; generic function call missing a type argument; unsafe pointer cast outside an <code>unsafe</code> block; type alias RHS does not evaluate to a type. Each instance carries a distinct human-readable message.
[E013]	Name error	Covers two sub-cases: (1) variable redeclaration or shadowing — a new <code>STMT_VAR</code> uses a name already bound in the current scope; (2) assignment to an undeclared identifier; (3) <code>main</code> is declared as <code>func</code> rather than <code>proc</code> .
[E014]	Non-exhaustive match / comptime error	Raised in two contexts: (1) a <code>case</code> expression or <code>match</code> statement does not cover all variants of the scrutinee type (exhaustiveness check in <code>src/sema/typecheck.h</code>); (2) a <code>comptime if</code> condition cannot be reduced to a compile-time constant; (3) CTFE recursion depth exceeds <code>COMPTIME_MAX_DEPTH = 256</code> (<code>src/sema/comptime.h</code>).
[E015]	Division by zero	Inside a <code>func</code> , VRA determines that the right-hand operand of a <code>/</code> or <code>%</code> expression may be zero (range includes 0, or range is fully unknown) (<code>src/sema/typecheck.h</code>).

.8 Termination-measure errors (E080–E082)

- ¹ Codes E080–E082 are emitted by the bounded-while verifier embedded in `src/sema.h`. They apply only to `while` loops that carry a `decreasing` measure clause.

Table 6: Termination diagnostic codes.

Code	Summary	Triggering condition
[E080]	Measure non-negativity failure	The verifier cannot prove that the termination measure is ≥ 0 when the loop condition is true. Specifically, no non-negativity pattern matches: the measure is not of the form $b - a$ with $b \geq a$ via an in-guard, not a single variable with a proven ≥ 0 range, and not reducible by and-chain traversal.

(continued)

Code	Summary	Triggering condition
[E081]	No variables in measure	<code>measure_extract_vars</code> returns an empty variable set. This means the measure expression contains no identifiers and cannot decrease as a function of any program variable.
[E082]	Measure decrease not provable	The verifier cannot confirm that the body strictly decreases the measure. Either no assignment to any measure variable is found in the body, or all found assignments have <code>ASSIGN_DIRECTION_UNKNOWN</code> or <code>ASSIGN_DIRECTION_INCREASE</code> direction.

.9 Parser errors (E100)

Table 7: Parser diagnostic codes.

Code	Summary	Triggering condition
[E100]	Parse error	All parser errors share this code. Specific messages include: semicolons not allowed (Lain uses newlines as statement separators); unexpected token at top level (an unrecognised declaration keyword); unexpected token in expression (primary expression parse failed). The parser calls <code>parser_error()</code> which prints the message and calls <code>exit(1)</code> (src/parser/core.h).

.10 VRA bounds diagnostics

- ¹ The Value-Range Analysis bounds checker ([src/sema/bounds.h](#)) does not use the [E`XXXX`] code scheme. It prefixes messages with [VRA] and emits one of four conditions:

Table 8: VRA bounds-check messages.

Message text (pre-fix)	Condition
index may be negative	The index expression's <code>Range.min</code> is < 0 .
index N out of bounds	Both index and length are statically known and $\text{index} \geq \text{length}$.
index range unknown ...	The index range is unknown and the array length is known; a runtime bounds check is emitted into the C output.
cannot prove index $< \text{length}$	The index range is known but the array length is unknown; a runtime bounds check is emitted.

(continued)

Message text (pre-fix)	Condition
both index and length unknown	Neither bound is known at compile time; a runtime bounds check is emitted. This is the common case for dynamically-sized slices with non-trivial index expressions.

- 2 The first two conditions are hard errors (exit code 1). The last three are informational: they indicate that a runtime check has been inserted, not that an error has been found. However, the compiler may be configured to treat unproved accesses as warnings in future versions; currently the distinction is only visible in the diagnostic prefix.

.11 Diagnostic format

- 1 All structured diagnostics follow the format:

```
1 [EXXX] Error Ln <line>, Col <col>: <message>
```

- 2 For diagnostics that also invoke `diagnostic_show_line()`, the output continues with two additional lines:

```
1 <source line text>
2 <caret string>
```

where the caret string contains a `^` aligned under the column position. The function `diagnostic_show_line()` is declared in [src/sema.h](#) and called at approximately 68 sites across the semantic analysis passes.

- 3 The VRA messages and the older borrow-checker messages in [src/sema/region.h](#) do not yet include the `Ln/Col` prefix on all call sites and do not invoke `diagnostic_show_line()`. Unifying these is a known area for future improvement.

Pass Interactions

- 4 The Lain compiler’s semantic analysis is not a sequence of fully independent passes that each transform the AST and hand it off to the next. Instead it is two *phases* — resolve and walk — orchestrated by a single function `sema_resolve_module` in `src/sema.h`, with tightly coupled sharing of mutable global state between the analysis components within each phase. This annex describes how those components interact with one another and with the shared state, and how the two-phase structure enforces the dependency ordering between analyses.

.12 Two-phase structure

- 1 The boolean flag `sema_walk_phase` (declared in `src/sema.h`) partitions all semantic analysis into two epochs per function body:
 1. **Resolve phase** (`sema_walk_phase = false`). `sema_resolve_stmt` is called on every statement. Name resolution, UFCS desugaring, generic instantiation, `comptime if` branch selection, and the initial call to `sema_infer_expr` all occur here. VRA range propagation, bounds checks, and linearity checks are *not* performed.
 2. **Walk phase** (`sema_walk_phase = true`). `walk_stmt` is called on every statement a second time. This invokes `sema_infer_expr` again, which now sees all `->type` annotations established during resolve, and activates VRA range propagation, in-guard tracking, bounds checking, and the termination-measure verifier.
- 2 The guard idiom used by walk-only checks is:

```
1 if (sema_walk_phase) {  
2     /* bounds check, VRA check, etc. */  
3 }
```

This prevents false positives during resolve, when type information may still be incomplete (e.g. a generic is being instantiated, or a forward reference is being resolved).

.13 Shared global state

- 1 All semantic passes share the following global tables, declared in `src/sema.h` before the sub-header includes:

Global	Type	Consumers
<code>sema_globals</code>	HashTable (4096 buckets)	Resolve, typecheck
<code>sema_locals</code>	HashTable (4096 buckets)	Resolve, typecheck
<code>sema_scope</code>	ScopeFrame	Resolve, linearity
<code>sema_ranges</code>	RangeTable *	VRA, bounds
<code>sema_in_guards</code>	InGuardEntry *	VRA, bounds
<code>sema_ltable</code>	LTable *	Linearity
<code>sema_btable</code>	BorrowTable *	Borrows
<code>sema_source_text</code>	const char *	Diagnostics
<code>sema_source_file</code>	const char *	Diagnostics

- 2 The symbol tables (`sema_globals`, `sema_locals`) are written by the resolver (name binding and UFCS desugaring) and read by the typechecker (callee lookup, struct field lookup). The range table (`sema_ranges`) is written by VRA and read by the bounds checker. The in-guard table (`sema_in_guards`) is written by `sema_push_in_guards` and read by the bounds checker. The linearity table (`sema_ltable`) and borrow table (`sema_btable`) are written and read exclusively by their respective passes.

.14 Resolve phase interactions

- 1 Within the resolve phase the call graph is:
1. `sema_resolve_stmt` dispatches on `StmtKind`. For every expression inside the statement it calls `sema_resolve_expr`, which handles UFCS transformation and generic call expansion.
 2. `sema_resolve_expr` calls `sema_infer_expr` on any sub-expression whose type is needed for resolution decisions (e.g. the receiver type of a member call is needed to determine the UFCS candidate). This creates a partial re-entrant dependency between resolve and typecheck.
 3. For generic calls (`EXPR_CALL` on a comptime-parameterised `DECL_FUNCTION`), `sema_resolve_expr` calls into `src/sema/comptime.h` to evaluate comptime arguments, then calls `src/sema/generic.h` to clone and substitute the function body, then recursively calls `sema_resolve_module` on the specialisation. This means generic instantiation is fully recursive and nested within resolve.
 4. For `STMT_COMPTIME_IF`, `sema_resolve_stmt` evaluates the condition via `comptime_evaluate_expr`, sets `is_taken`, and recursively resolves only the selected branch. The unselected branch is never visited.

.15 Walk phase interactions

- 1 The walk phase processes each statement in the same statement-list order as resolve. `walk_stmt` calls `sema_infer_expr`, which in turn calls the following analyses:

Analysis	Entry point	Called from
Type inference	<code>sema_infer_expr</code>	<code>walk_stmt</code>
VRA propagation	<code>range_set/range_eval</code>	Inside <code>sema_infer_expr</code>
In-guard push	<code>sema_push_in_guards</code>	<code>walk_stmt</code> (STMT_WHILE/STMT_IF)
Bounds check	<code>sema_check_bounds</code>	Inside <code>sema_infer_expr</code> (EXPR_INDEX)
Linearity check	<code>linearity_check_stmt</code>	<code>walk_stmt</code> (after infer)
Borrow check	<code>borrow_register</code>	Inside <code>linearity_check_stmt</code>
Termination verify	bounded-while verifier	<code>walk_stmt</code> (STMT_WHILE with measure)

- 2 The ordering within a single walk-phase statement is significant:
 1. `sema_infer_expr` runs first. It populates `->type` on the expression tree and propagates range information into `sema_ranges`. Bounds checks are triggered as a side-effect of inferring `EXPR_INDEX` expressions.
 2. In-guards are pushed immediately before the loop/if body is walked. The guard for a condition `idx in arr` is added to `sema_in_guards` so that bounds checks on `arr[idx]` inside the body see the proof.
 3. Linearity checking (`linearity_check_stmt`) runs after type inference for the same statement, so it sees the `->type` annotations needed to determine linearity. Borrow registration is a sub-step of linearity checking: shared borrows are registered when a variable is passed to a shared parameter, mutable borrows when passed to a `var` parameter.
 4. The bounded-while verifier is invoked at `STMT_WHILE` nodes that carry a non-null `measure` field. It reads the current `sema_ranges` state to evaluate non-negativity and reads the condition subtree to extract in-guard patterns for the decrease proof.

.16 VRA and bounds interaction

- 1 VRA (chapter 14) and the bounds checker (chapter 15) share the `sema_ranges` table and the `sema_in_guards` linked list. Their interaction is:
 1. During `sema_infer_expr` for `EXPR_BINARY` assignment, `range_set` updates the range of the target variable.
 2. When `sema_infer_expr` encounters `EXPR_INDEX`, it reads the range of the index expression via `range_eval` and calls `sema_check_bounds`.
 3. `sema_check_bounds` first checks the in-guard table for a proven (`index`, `container`) pair. If found, it returns without emitting a runtime check. If not, it queries `sema_ranges` for the range. Static ranges trigger either a compile-time error or a silent pass; unknown ranges trigger emission of a C runtime bounds-check expression.
- 2 This means VRA must run before the bounds checker for any given index expression, and the in-guard table must be populated before the loop body is walked. Both of these orderings are guaranteed by the sequential evaluation inside `sema_infer_expr` and the explicit push-before-body structure in `walk_stmt`.

.17 Linearity and borrows interaction

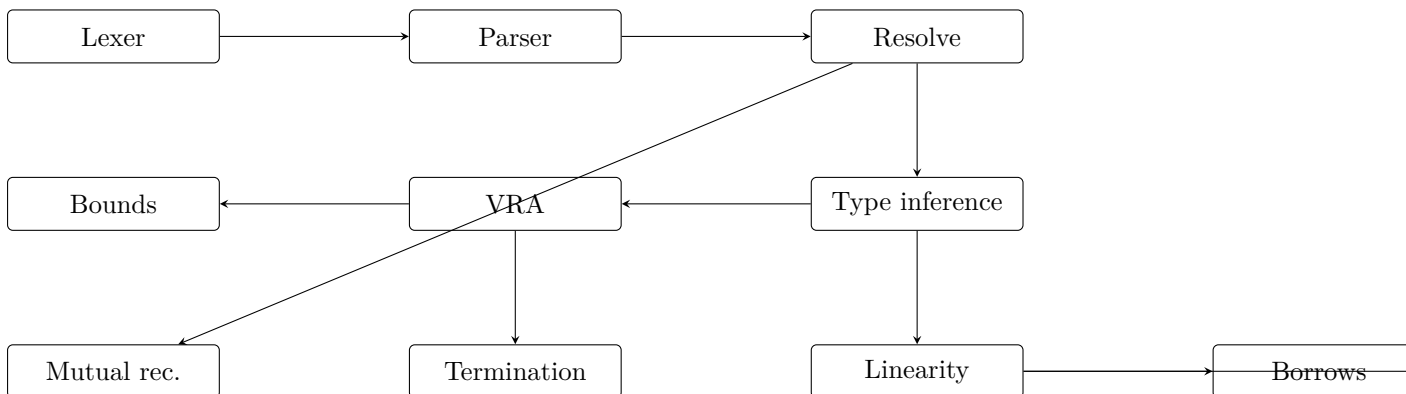
- 1 The linearity checker (chapter 12) and the borrow checker (chapter 13) share `sema_ltable` and `sema_btable` but do not call each other directly. Their interaction is through the following read/write protocol:
 - When the linearity checker encounters a move (`EXPR_MOVE` or an implicit transfer into a `MODE_OWNED` parameter), it calls `borrow_is_live(sema_btable, id)` to verify no borrow is active before marking the variable consumed. If a borrow is live, `[E008]` is emitted.
 - When a variable is passed to a shared (`MODE_SHARED`) parameter, the linearity checker calls `borrow_register` with `MODE_SHARED` and registers the two-phase `RESERVED` borrow. After all arguments are evaluated, `borrow_promote_to_active` upgrades the borrow to `ACTIVE`.
 - When a variable is passed to a mutable (`MODE_MUTABLE`) parameter, the linearity checker calls `borrow_register` with `MODE_MUTABLE`. The borrow checker verifies no conflicting borrow exists (`[E004]`) before registering.
 - At scope exit, the linearity checker scans `sema_ltable` for unconsumed linear variables (`[E003]`), and the borrow checker automatically expires borrows whose region no longer contains the current statement index.

.18 Mutual recursion detection

- 1 Mutual recursion detection (the DFS described in chapter 9) runs as a standalone pre-pass over the resolved declaration list, after both phases have completed. It does not share state with the other analyses except reading the resolved `->decl` pointers set during resolve. Its output is purely diagnostic: `[E011]` is emitted and the exit code is set if a cycle involving a `func` is found.

.19 Pass dependency DAG

- 1 The dependency relationships between passes can be summarised as a directed acyclic graph:



- 2 Edges represent “must run before” ordering. All arcs involving the walk-phase analyses (VRA, bounds, linearity, borrows, termination) are implicitly conditioned on the walk phase having begun (`sema_walk_phase = true`).

Declaration Flow

- 3 This annex traces the lifecycle of a Lain declaration from source text to emitted C, following a single **func** declaration as the running example. The same pipeline applies to **proc**, **struct**, **enum**, **import**, **extern func**, **extern type**, and **type** aliases, with per-kind differences noted where they diverge.

.20 Stage 1: Lexing

- 1 The lexer ([src/lexer.h](#)) emits a stream of `Token` values. For a declaration, the relevant tokens are:
- `TOKEN_KEYWORD_FUNC` or `TOKEN_KEYWORD_PROC` — the declaration introducer.
 - `TOKEN_IDENTIFIER` — the function name.
 - `TOKEN_LPAREN` / `TOKEN_RPAREN` — parameter list delimiters.
 - Type-name identifiers and type-qualifier keywords (`TOKEN_KEYWORD_MUT`, `TOKEN_KEYWORD_MOV`) for parameters and the return type.
 - `TOKEN_LBRACE` / `TOKEN_RBRACE` — body delimiters.
- 2 The lexer does not attach meaning to any token. The stream is consumed on-demand by the parser via `lexer_next()` calls.

.21 Stage 2: Parsing

- 1 `parse_decl` ([src/parser/decl.h](#)) dispatches on the leading keyword. For `TOKEN_KEYWORD_FUNC`, it calls `parse_func_proc_decl_impl` with `is_proc = false`.
- 2 The parser allocates a `Decl` node on `ast_arena` via `ARENA_PUSH_STRUCT`. It fills in:
- `kind = DECL_FUNCTION` (or `DECL_PROCEDURE`).
 - `name` — the identifier string (pointer into source, zero-copy).
 - `as.function_decl.params` — a `DeclList *` of `DECL_VARIABLE` nodes, one per parameter.
 - `as.function_decl.return_type` — a `Type *` constructed from the declared return type tokens.
 - `as.function_decl.body` — a `StmtList *` of body statements, recursively produced by `parse_stmt_list`.
 - `line, col` — source position of the declaration keyword.
- 3 For **extern func**, `parse_extern_func_proc_decl_impl` is called. It produces a `DECL_EXTERN_FUNCTION` node with `body = NULL`. For **struct**, `parse_struct_decl` fills `as.struct_decl.fields`. For **import**, `parse_import_decl` fills `as.import_decl.path`.

.22 Stage 3: Module loading

- 1 For `DECL_IMPORT` nodes, `sema_resolve_module` ([src/sema.h](#)) calls `load_module` ([src/module.h](#)).
`load_module`:
 1. Translates the dotted import path to a file path.
 2. Reads and lexes the file.
 3. Recursively calls the parser to produce a `DeclList *`.
 4. Splices the loaded declarations in-place into the caller's declaration list, replacing the `DECL_IMPORT` node.
 5. Marks the module path in the loaded-modules list to prevent cycles.
- 2 After this stage, the `DeclList *` seen by the rest of `sema` contains only flattened, non-import declarations.

.23 Stage 4: Resolve phase

- 1 `sema_resolve_module` iterates the declaration list twice:

Forward-declaration scan. On the first pass, every top-level `DECL_FUNCTION`, `DECL_PROCEDURE`, `DECL_STRUCT`, `DECL_ENUM`, `DECL_EXTERN_FUNCTION`, `DECL_EXTERN_PROCEDURE`, `DECL_EXTERN_TYPE`, and `DECL_VARIABLE` is inserted into `sema_globals` ([src/sema/resolve.h](#) lines 128–200):

- Functions: a `Symbol` with `kind = SYM_FUNCTION` and a pointer back to the `Decl` node.
- Struct/enum types: a `Symbol` with `kind = SYM_TYPE`.
- Global variables: a `Symbol` with `kind = SYM_VARIABLE`.

This forward scan enables mutual calls between functions without requiring forward declarations in user code.

Body resolution. On the second pass, `sema_resolve_decl` visits each declaration. For function bodies, `sema_resolve_stmt` is called on every body statement. During body resolution:

- Parameter `DECL_VARIABLE` nodes are inserted into `sema_locals`.
- Identifiers in expressions are resolved by `sema_lookup` (locals first, then globals), and the `->decl` pointer on `Expr` nodes is set.
- UFCS desugaring rewrites `EXPR_MEMBER` calls to prepend the receiver as the first argument.
- Generic calls are expanded: comptime arguments are evaluated, the body is deep-cloned, type parameters are substituted, and the specialisation is recursively resolved and inserted into `sema_globals`.
- `comptime if` branches are evaluated and the non-taken branch is pruned.

.24 Stage 5: Walk phase

- 1 After body resolution, `sema_walk_phase` is set to true and `walk_stmt` is called on each body statement. In this phase:
 - `sema_infer_expr` propagates types bottom-up, annotating every `Expr` node's `->type` field with the inferred `Type *`. Integer widening is applied at binary operations.
 - VRA propagates integer ranges for every identifier and expression.
 - In-guard proofs are established at `while` / `if` conditions.
 - Bounds checks are performed at every `EXPR_INDEX` node.
 - The linearity checker tracks consumption state for linear variables, enforcing E001–E008 and E003 at scope exit.
 - The borrow checker enforces aliasing rules (E004) and lifetime constraints (E010).
 - The bounded-while verifier checks termination measures (E080–E082).

.25 Stage 6: Mutual recursion check

- 1 After all declarations have been resolved and walked, the mutual recursion DFS (`src/sema.h`) scans the resolved call graph for cycles. Any cycle involving a `func` declaration triggers [E011].

.26 Stage 7: Code emission

- 1 The emitter (`src/emit/core.h`) iterates the fully-resolved and walk-annotated `DeclList *` in six sub-passes:
 1. **Runtime header** — `src/emit/lain_header.h` is written verbatim.
 2. **c_include directives** — `DECL_C_INCLUDE` nodes are emitted as `#include` lines.
 3. **Type forward declarations** — `src/emit/type_order.h` topologically sorts struct and enum types and emits forward `typedef struct` and `typedef enum` declarations to satisfy C's requirement that types be declared before use.
 4. **Struct/enum definitions** — full `typedef struct {...}` and `typedef enum {...}` bodies are emitted. For enums with payload variants, a tag enum and a tagged-union struct are emitted. Constructor functions are emitted by `src/emit/ctor.h`.
 5. **Function prototypes** — `emit_forward_decl` (`src/emit/decl.h`) emits a C prototype for every `DECL_FUNCTION`, `DECL_PROCEDURE`, `DECL_EXTERN_FUNCTION`, and `DECL_EXTERN_PROCEDURE`. This allows the function bodies to reference each other freely.
 6. **Function bodies** — `emit_decl` emits the full C function definition for every `DECL_FUNCTION` and `DECL_PROCEDURE`. Extern declarations are emitted as prototypes only (no body).

.27 Per-kind summary

Table 9: Declaration lifecycle by kind.

DeclKind	Resolve action	Emit action
DECL_FUNCTION DECL_PROCEDURE	/ Symbol inserted into <code>sema_globals</code> ; body resolved and walked; mutual recursion checked.	Type forward decl (if needed), prototype, full body via <code>emit_decl</code> .
DECL_EXTERN_FUNCTION / DECL_EXTERN_PROCEDURE	Symbol inserted into <code>sema_globals</code> ; no body to resolve.	C <code>extern</code> prototype only.
DECL_STRUCT	Symbol inserted into <code>sema_globals</code> ; field types resolved.	Forward typedef, then full typedef <code>struct {...}</code> .
DECL_ENUM	Symbol inserted into <code>sema_globals</code> ; variant types resolved.	Forward typedef, tag enum, tagged-union struct, constructor functions.
DECL_IMPORT DECL_EVAL_IMPORT	/ Replaced in-place by the loaded module's <code>DeclList *</code> before resolve.	Not emitted directly; replaced declarations are emitted.
DECL_C_INCLUDE	No sema action.	Emitted as <code>#include "..."</code> in the includes section.
DECL_EXTERN_TYPE	Symbol inserted into <code>sema_globals</code> as <code>SYM_TYPE</code> .	Emitted as a forward C typedef or opaque struct.
DECL_TYPE_ALIAS	Comptime RHS evaluated; result stored in <code>DeclTypeAlias.resolved_type</code> .	Emitted as a C typedef.
DECL_VARIABLE	If top-level: inserted into <code>sema_globals</code> . If parameter: inserted into <code>sema_locals</code> during body resolve.	Top-level variables emitted as C global declarations; parameters emitted as part of their function signature.
DECL_DESTRUCT	Expanded during resolve into a hidden <code>_param_N</code> parameter plus assignment statements for each field.	Not directly emitted; expansion is emitted as assignments.

Index

- abstract syntax tree, 21
- aliasing rules, 60
- architecture, 3
- arena allocator, 4, 9
- ASCII classification, 11
- assignment statements, 44
- AST, 21
 - deep clone, 27
 - reference, 83
 - semantic annotation, 27
- borrow checking, 59
- BORROW_ACTIVE, 60
- BORROW_RESERVED, 60
- BorrowEntry, 59
- bounded while, 69
- bounds checking, 65
- build, 7
- C99 backend, 75
- cast expressions, 41
- code generation, 75
- comments
 - nested block, 16
- compile-time evaluation, 73
- comptime, 73
- comptime if, 74
- comptime_evaluate_expr, 73
- ComptimeEnv, 73
- constraint propagation, 64
- consumption rules, 58
- Decl node, 22
- declaration flow, 97
- declarations
 - lifecycle, 97
- DeclKind, 22
- decreasing measure, 69
- defer, 76
- design philosophy, 3
- destructuring parameters, 53
- diagnostics
 - catalogue, 87
 - format, 91
- enum emission, 76
- enum exhaustiveness, 67
- error codes, 87
- exhaustiveness checking, 67
- Expr node, 25
- ExprKind, 25
- field-level consumption, 58
- file I/O, 10
- for statement, 45
- generic functions
 - instantiation, 73
- generics, 73
- global state, 7
 - semantic, 93
- Id struct, 21
- identifier resolution, 53
- if statement, 44
- in-guard table, 64
- infrastructure, 9
- InGuardEntry, 64
- integer widening, 55
- keywords
 - classification, 16
- LEntry, 57
- lexer, 13
 - FSM, 14
 - lookahead, 16
- LexerState, 14
- linear types, 57
- linearity, 57
 - interaction with borrows, 96
- linearity errors, 87
- list nodes, 27
- load_module, 30
- LTable, 57

- match statement, 45
- memory
 - arenas, 4
- module loading, 29
- module name
 - path conversion, 29
- module system, 29
 - cycles, 30
- ModuleNode, 29
- mutual recursion
 - detection, 96
- mutual recursion detection, 51
- name resolution, 53
- naming conventions, 6
- NLL, 58, 61
- Non-Lexical Lifetimes, 58
- numeric literals
 - lexing, 15
 - parsing, 37
- operator precedence, 36
- ownership, 57
- OwnershipMode, 21
- panic assertions, 11
- parser
 - declarations, 43, 45
 - error reporting, 36
 - expressions, 39
 - framework, 35
 - macros, 36
 - path expressions, 37
 - statements, 43
 - token advancement, 35
- parser errors, 90
- Parser struct, 35
- pass dependency, 96
- pass interactions, 93
- pattern matching
 - exhaustiveness, 67
- pipeline, 3
 - compilation stages, 3
- platform constants, 54
- Pratt parser, 39
- precedence climbing, 39
- primary expressions, 40
- primitive types, 9
- purity errors, 88
- range refinement, 64
- Range struct, 63
- RangeTable, 63
- Region struct, 59
- regions, 59
- resolve phase, 94
- saturating arithmetic, 63
- scope
 - block scope, 50
- scope building, 54
- sema_infer_expr, 55
- sema_resolve_expr, 53
- sema_resolve_module, 49
- sema_type_is_linear, 57
- sema_walk_phase, 93
- semantic analysis
 - orchestration, 49
 - passes, 93
 - two phases, 49
- source files, 5
- standard library, 79
- static bounds checking, 65
- std.c, 79
- std.fs, 79
- std.io, 79
- std.math, 79
- std.option, 80
- std.result, 80
- std.string, 80
- Stmt node, 24
- StmtKind, 24
- string literals
 - lexing, 16
- string type, 11
- Symbol struct, 50
- symbol table, 50
- termination errors, 89
- termination measure, 69
- termination verification, 69
- test categories, 81
- test file conventions, 81
- test suite, 81
- testing infrastructure, 81
- Token struct, 13
- tokeniser, 13
- TokenKind, 13
- two-phase borrows, 59, 60
- type aliases, 74
- type compatibility, 55
- type emission, 75
- type errors, 88
- type inference, 55
- Type node, 21

TypeKind, 21

UFCS, 53

unary expressions, 40

Uniform Function Call Syntax, 53

value range analysis, 63

var statement, 44

VRA, 63

 diagnostics, 90

 interaction with bounds, 95

walk phase, 94

walk_stmt, 51

while statement, 45

wildcard arm, 67